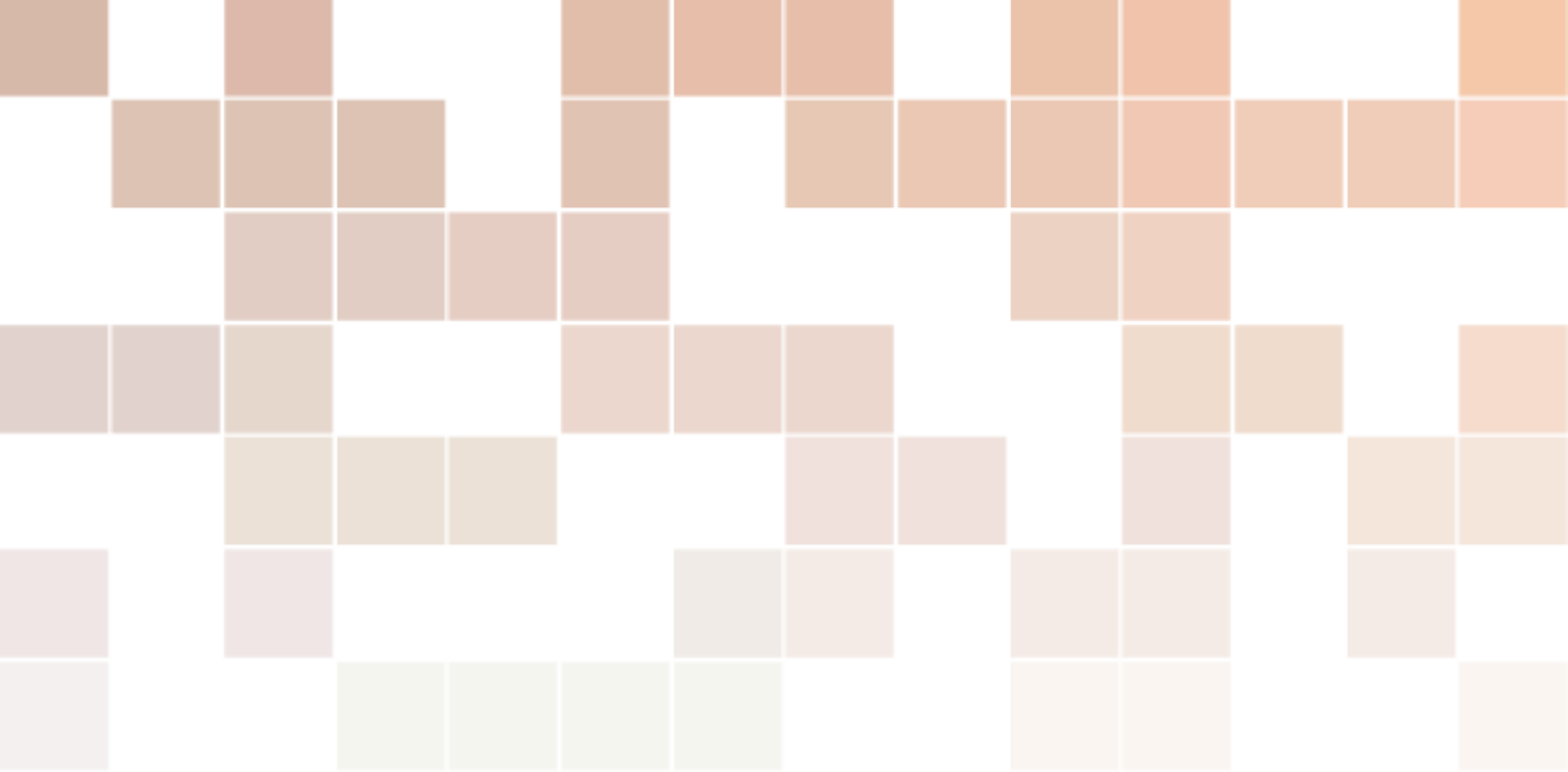
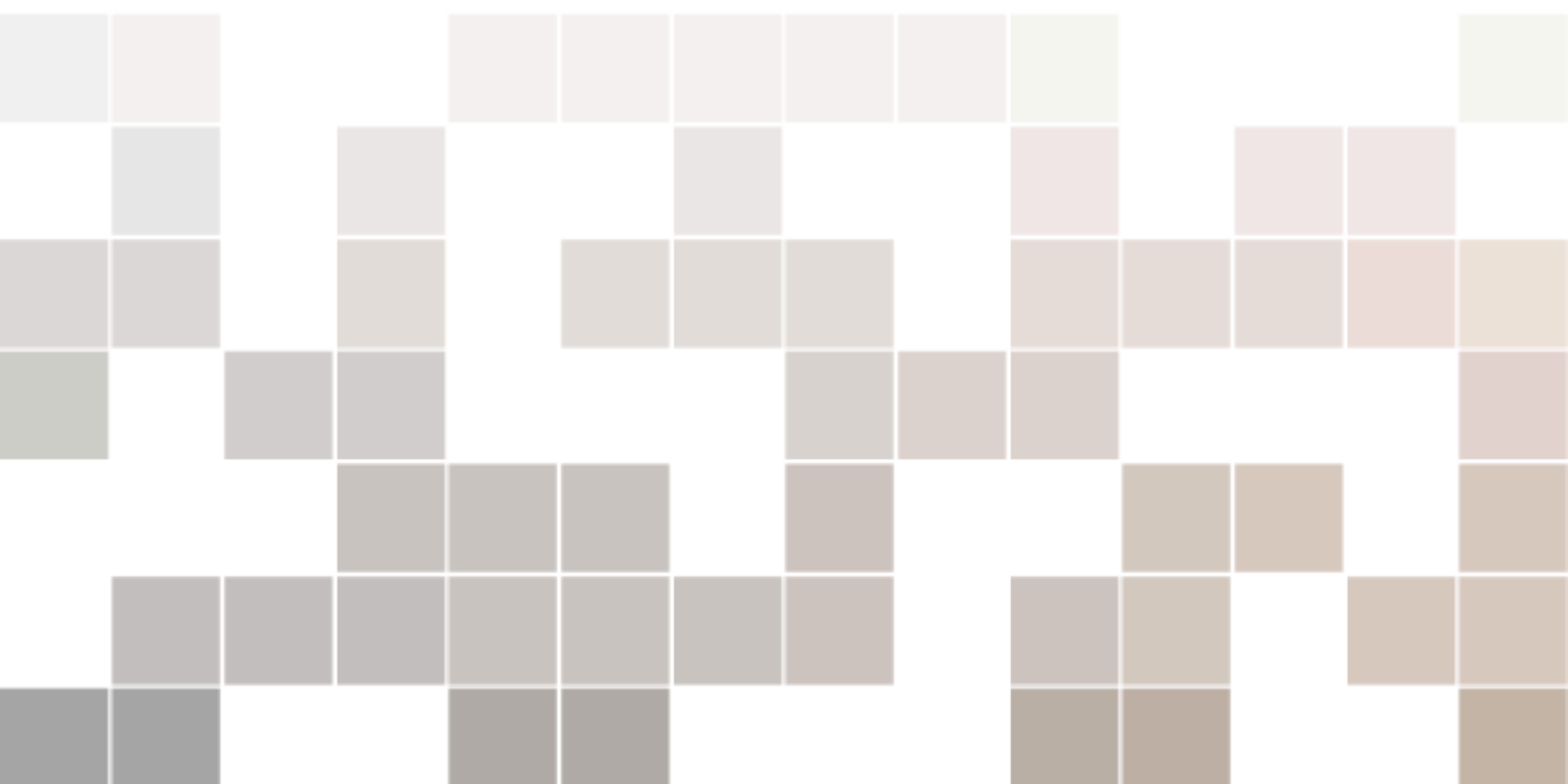




强化学习和多模态教程

理论与实践

左元



目录

I

第一部分

1	Vision Transformer	13
1.1	导入库和模块	14
1.2	补丁嵌入 (Patch Embedding)	14
1.3	类别对应的 token 和位置编码	16
1.4	注意力头	18
1.5	多头注意力	20
1.6	Transformer 编码器	21
1.7	Vision Transformer	22
1.8	训练参数	24
1.9	加载 MNIST 数据集	24
1.10	训练循环	25
1.11	评估模型	25
2	CLIP	27
2.1	导入库和模块	27
2.2	图像和文本编码器	28
2.2.1	位置嵌入	28
2.2.2	注意力头	28
2.2.3	多头注意力	29
2.2.4	Transformer 编码器	29
2.2.5	文本的分词器	30
2.2.6	文本编码器	31
2.2.7	图像编码器	33
2.3	CLIP 模型	34
2.3.1	数据	37
2.3.2	训练参数	39
2.3.3	加载数据集	40
2.3.4	训练模型	40

2.3.5	评估模型	41
2.3.6	零样本分类	42
3	扩散模型	45
3.1	通俗讲解	45
3.1.1	扩散	45
3.1.2	扩散模型的架构	46
3.1.3	模型设计	48
3.2	扩散模型理论简介	50
3.2.1	概述	50
3.2.2	正向扩散过程	50
4	Sectioning Examples	51
4.1	Section Title	51
4.1.1	Subsection Title	51
5	In-text Element Examples	55
5.1	Referencing Publications	55
5.2	Link Examples	55
5.3	Lists	55
5.3.1	Numbered List	55
5.3.2	Bullet Point List	55
5.3.3	Descriptions and Definitions	55
5.4	International Support	56
5.5	Ligatures	56
5.6	Referencing Chapters	56

II

Part Two Title

6	Mathematics	59
6.1	Theorems	59
6.1.1	Several equations	59
6.1.2	Single Line	59
6.2	Definitions	59
6.3	Notations	60
6.4	Remarks	60
6.5	Corollaries	60
6.6	Propositions	60
6.6.1	Several equations	60
6.6.2	Single Line	60
6.7	Examples	60
6.7.1	Equation Example	60
6.7.2	Text Example	60
6.8	Exercises	60
6.9	Problems	61
6.10	Vocabulary	61

7	Presenting Information and Results with a Long Chapter Title .	63
7.1	Table	63
7.2	Figure	63
	参考文献	65
	Index	67
	Appendices	69
A	Appendix Chapter Title	69
A.1	Appendix Section Title	69
B	Appendix Chapter Title	71
B.1	Appendix Section Title	71

List of Figures

1.1 Vit 架构图	13
1.2 补丁的例子	14
1.3 将图像转换为补丁：第一步	15
1.4 将图像转换为补丁：第二步	16
1.5 将图像转换为补丁：第三步	16
1.6 位置编码的作用	17
1.7 左图：使用 Conv2D 运算分成 16 个 8x8 块的 32x32 MNIST 图像。右图：添加位置编码和类别 token 后的 16 个图像补丁，使用随机数据初始化。	18
1.8 注意力机制	19
2.1 clip 原理，来自原始论文	27
2.2 注意力分数的掩码	29
2.3 分词过程	31
2.4 余弦相似度的计算	36
2.5 复制掩码	38
2.6 因果注意力分数的掩码	39
3.1 水滴的扩散过程	45
3.2 高清图像的扩散过程	46
3.3 从高斯分布中采样一个随机值	46
3.4 预测图像与实际图像之间的差异使用损失函数计算	47
3.5 将噪声作为预测目标	48
3.6 U-net 架构（以最低分辨率为 32x32 像素为例）。每个蓝色方框对应一个多通道特征图。方框顶部标注了通道数量。方框左下角标注了 x-y 尺寸。白色方框表示复制的特征图。箭头表示不同的操作。	49
3.7 共享模型	49
3.8 正向扩散和逆向扩散	50
7.1 Figure caption.	63
7.2 Floating figure.	64



List of Tables

7.1	Table caption.	63
7.2	Floating table.	64



第一部分

1	Vision Transformer	13
1.1	导入库和模块	14
1.2	补丁嵌入 (Patch Embedding)	14
1.3	类别对应的 token 和位置编码	16
1.4	注意力头	18
1.5	多头注意力	20
1.6	Transformer 编码器	21
1.7	Vision Transformer	22
1.8	训练参数	24
1.9	加载 MNIST 数据集	24
1.10	训练循环	25
1.11	评估模型	25
2	CLIP	27
2.1	导入库和模块	27
2.2	图像和文本编码器	28
2.3	CLIP 模型	34
3	扩散模型	45
3.1	通俗讲解	45
3.2	扩散模型理论简介	50
4	Sectioning Examples	51
4.1	Section Title	51
5	In-text Element Examples	55
5.1	Referencing Publications	55
5.2	Link Examples	55
5.3	Lists	55
5.4	International Support	56
5.5	Ligatures	56
5.6	Referencing Chapters	56

1. Vision Transformer

🔥 提示

ViT: Vision Transformer

基于自注意力的 Transformer 模型由 Vaswani 等人在 2017 年的论文《Attention Is All You Need》中首次提出，并已广泛应用于自然语言处理中。Transformer 模型是 OpenAI 用来创建 ChatGPT 的模型。Transformer 不仅适用于文本，还适用于图像，基本上可以处理任何序列数据。2021 年，Dosovitsky 等人在他们的论文《An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale》中引入了将 Transformer 用于计算机视觉任务（例如图像分类）的想法。在他们的论文中，与卷积网络相比，他们的 Vision Transformer 模型能够取得出色的结果，并且需要更少的资源来训练。

在本教程中，我们将从头开始构建一个 Vision Transformer 模型，并在 MNIST 数据集上进行测试。

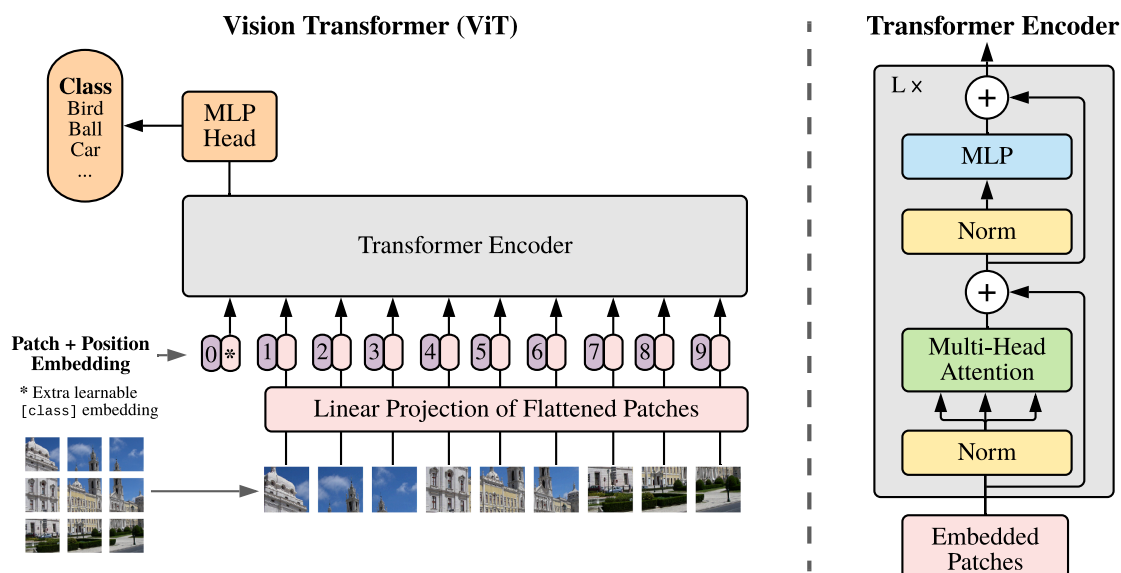


图 1.1 ViT 架构图

1.1 导入库和模块

本章所需依赖

Python

```

1 import torch
2 import torch.nn as nn
3 import torchvision.transforms as T
4 from torch.optim import Adam
5 from torchvision.datasets.mnist import MNIST
6 from torch.utils.data import DataLoader
7 import numpy as np

```

1.2 补丁嵌入 (Patch Embedding)

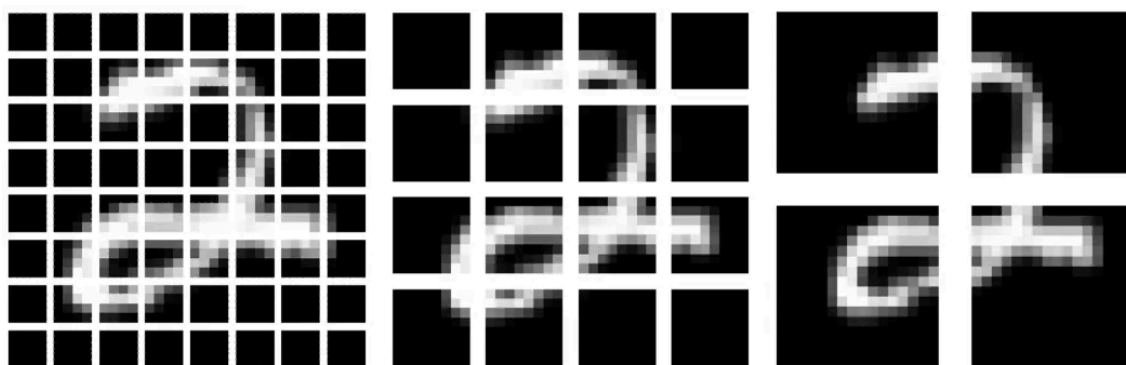


图 1.2 补丁的例子

补丁嵌入类

Python

```

1 class PatchEmbedding(nn.Module):
2     def __init__(
3         self,
4         d_model,    # 模型的维度
5         img_size,   # 图片大小
6         patch_size, # 补丁大小
7         n_channels  # 通道数量
8     ):
9         super().__init__()
10
11         self.d_model = d_model
12         self.img_size = img_size
13         self.patch_size = patch_size
14         self.n_channels = n_channels
15
16         self.linear_project = nn.Conv2d(
17             self.n_channels, # in_channels
18             self.d_model,   # out_channels
19             kernel_size=self.patch_size, # kernel_size
20             stride=self.patch_size # stride
21         )
22

```



```

23     # B: 批次大小
24     # C: 通道数量
25     # H: 图像高度
26     # W: 图像宽度
27     # P_col: 补丁的列
28     # P_row: 补丁的行
29     def forward(self, x):
30         x = self.linear_project(x) # (B, C, H, W) -> (B, d_model, P_col, P_row)
31         x = x.flatten(2) # (B, d_model, P_col, P_row) -> (B, d_model, P)
32         x = x.transpose(1, 2) # (B, d_model, P) -> (B, P, d_model)
33         return x

```

创建 Vision Transformer 的第一步是将输入图像拆分为补丁，并创建这些补丁的线性嵌入序列。我们能够通过使用 PyTorch 的 Conv2d 方法来实现这一点。

Conv2d 方法获取输入图像，将它们拆分为补丁，并提供大小等于 `d_model` 的线性投影。通过将 `kernel_size` 和步幅设置为补丁大小，我们确保补丁大小正确且没有重叠。

```

1 self.linear_project = nn.Conv2d(
2     self.n_channels,
3     self.d_model,
4     kernel_size=self.patch_size,
5     stride=self.patch_size
6 )

```

 Python

在 `forward` 方法中，我们通过 `linear_project/Conv2D` 方法传递具有形状 `(B, C, H, W)` 的输入，并输出形状 `(B, d_model, P_col, P_row)` 的张量。

```

1 def forward(self, x):
2     x = self.linear_project(x) # (B, C, H, W) -> (B, d_model, P_col, P_row)

```

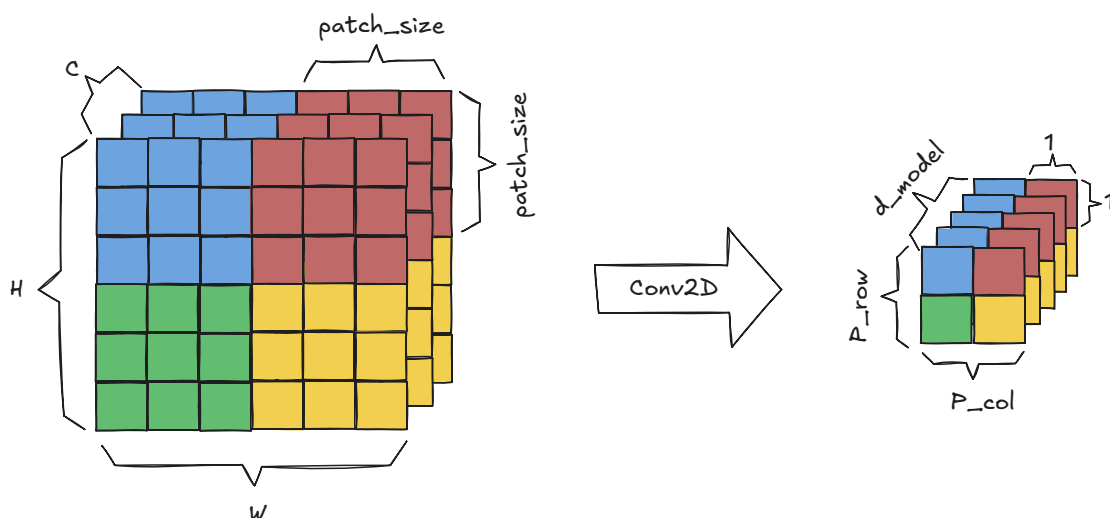
 Python


图 1.3 将图像转换为补丁：第一步

我们使用展平方法将补丁列和补丁行维度组合成一个补丁维度，从而得到 `(B, d_model, P)` 的形状

```

1 x = x.flatten(2) # (B, d_model, P_col, P_row) -> (B, d_model, P)

```

 Python

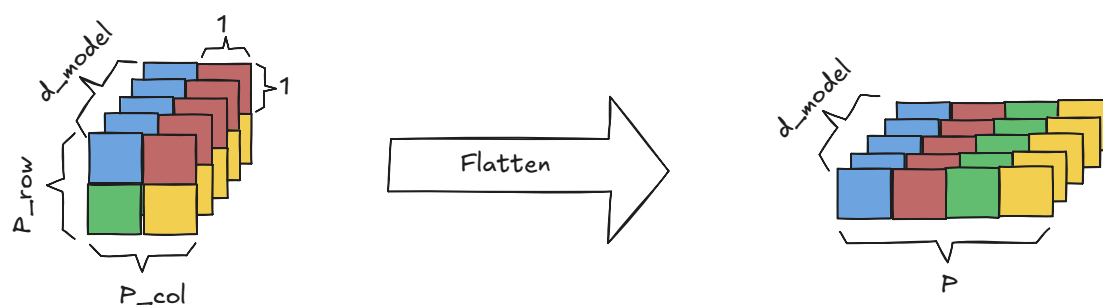


图 1.4 将图像转换为补丁：第二步

最后，我们使用转置方法切换 d_model 和补丁维度，得到 (B, P, d_model) 的形状。

```
1 x = x.transpose(1, 2) # (B, d_model, P) -> (B, P, d_model)
```

Python

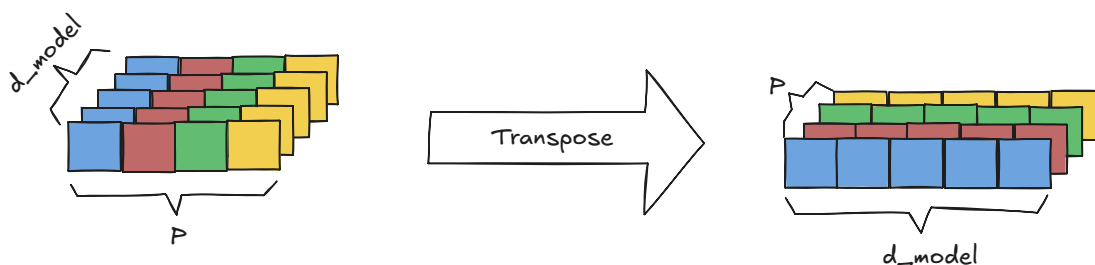


图 1.5 将图像转换为补丁：第三步

1.3 类别对应的 token 和位置编码

位置编码类

Python

```
1 class PositionalEncoding(nn.Module):
2     def __init__(self, d_model, max_seq_length):
3         super().__init__()
4         # 类别token
5         self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))
6         # 创建位置编码
7         pe = torch.zeros(max_seq_length, d_model)
8
9         for pos in range(max_seq_length):
10             for i in range(d_model):
11                 if i % 2 == 0:
12                     pe[pos][i] = np.sin(pos/(10000 ** (i/d_model)))
13                 else:
14                     pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/d_model)))
15         # 将位置编码固定
16         self.register_buffer("pe", pe.unsqueeze(0))
17
18     def forward(self, x):
19         # 为批次中的每张图片分配一个类别token
20         tokens_batch = self.cls_token.expand(x.size()[0], -1, -1)
```



```

21         # 将类别token添加到每个图像的补丁嵌入数组的开头
22         x = torch.cat((tokens_batch,x), dim=1)
23         # 将位置编码添加到嵌入中
24         x = x + self.pe
25         return x

```

ViT 模型使用向补丁嵌入添加可学习的类别 token 的标准方法来执行分类。

```

1 # class token: 类别或者分类对应的token
2 self.cls_token = nn.Parameter(torch.randn(1, 1, d_model))

```

Python

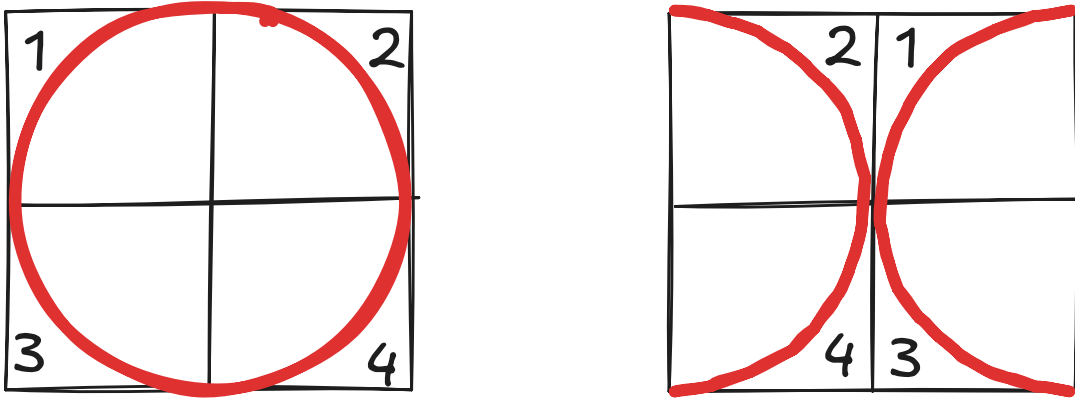


图 1.6 位置编码的作用

与 LSTM 等按顺序接受嵌入的模型不同，Transformer 并行的接受嵌入。虽然这提高了速度，但 Transformer 不知道序列的顺序是什么。这是一个很大的问题，因为改变序列的顺序很可能会改变其含义。上图就是一个例子，它显示更改图像的补丁顺序可以将图像从 O 更改为更接近 X 的图像。为了解决这个问题，需要将位置编码添加到嵌入中。每个位置编码对于它所表示的位置都是唯一的，这允许模型识别每个补丁嵌入应该在的位置。为了将位置编码添加到补丁嵌入中，它们必须具有相同的维度， d_{model} 。我们使用下面的公式获得位置编码。

定义 1.1 — 位置编码。

$$\begin{aligned}
 \text{PE}_{(\text{pos}, 2i)} &= \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \\
 \text{PE}_{(\text{pos}, 2i+1)} &= \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)
 \end{aligned} \tag{1.1}$$

```

1 # max_seq_length: 补丁嵌入的数量, d_model: 嵌入维度
2 pe = torch.zeros(max_seq_length, d_model)
3
4 for pos in range(max_seq_length):
5     for i in range(d_model):
6         if i % 2 == 0:
7             pe[pos][i] = np.sin(pos/(10000 ** (i/d_model)))
8         else:
9             pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/d_model)))
10 # 禁止pe更新, pe保存在了显存中, 不会被反向传播更新
11 self.register_buffer('pe', pe.unsqueeze(0))

```

Python

在 `forward` 方法中，输入是多个图像的一批补丁嵌入。例如， 32×32 的图像可以分解为 16 个 8×8 大小的补丁。在此 `max_seq_length` 中，需要为 $16+1=17$ 才能创建足够的位置嵌入，每个补丁一个，分类对应的 `token` 一个。

因此，我们需要使用 `expand` 函数才能使用 `self.cls_token` 为批处理中的每个图像创建分类对应的 `token`。注意力机制会将有关整个序列的信息编码到序列中的每个 `token` 中。由于每个 `token` 都受到其自身信息的偏见，因此分类对应的 `token` 会创建序列中所有 `token` 的独立的摘要信息。

```
1 def forward(self, x):
2     tokens_batch = self.cls_token.expand(x.size()[0], -1, -1)
```

Python

然后，使用 `torch.cat` 方法将这些分类 `token` 添加到每个补丁嵌入的开头。

```
1 x = torch.cat((tokens_batch, x), dim=1)
```

Python

位置编码在输出之前添加。

```
1 x = x + self.pe
2 return x
```

Python

这是添加分类 `token` 之前和之后的数据的样子：

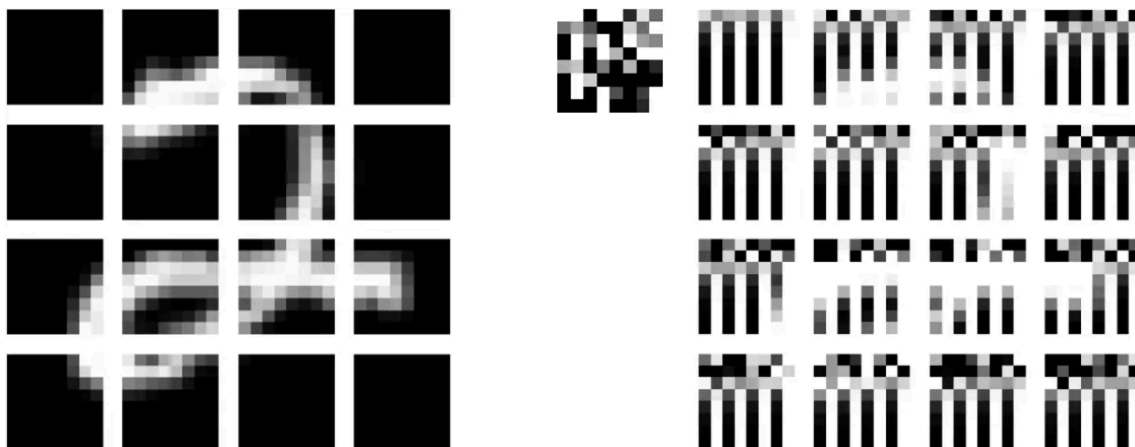


图 1.7 左图：使用 `Conv2D` 运算分成 16 个 8×8 块的 32×32 MNIST 图像。右图：添加位置编码和类别 `token` 后的 16 个图像补丁，使用随机数据初始化。

请注意，我们已经用 64 个卷积核初始化了 `Conv2D` 运算，每个卷积核中的每个补丁只占用一个像素，以免扭曲图像。

1.4 注意力头

注意力头

Python

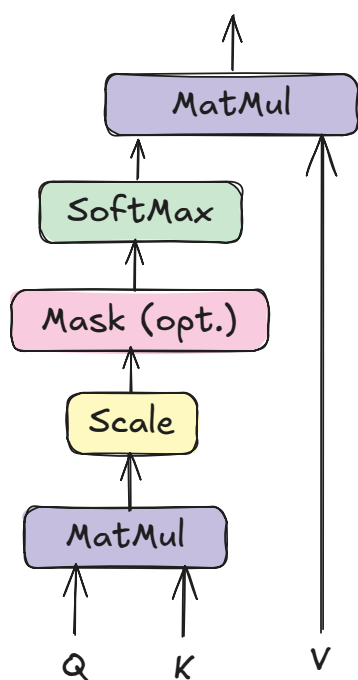
```
1 class AttentionHead(nn.Module):
2     def __init__(self, d_model, head_size):
3         super().__init__()
4         self.head_size = head_size
5
6         self.query = nn.Linear(d_model, head_size)
7         self.key = nn.Linear(d_model, head_size)
8         self.value = nn.Linear(d_model, head_size)
9
10    def forward(self, x):
11        # 计算Q, K, V
```

```

12     Q = self.query(x)
13     K = self.key(x)
14     V = self.value(x)
15
16     # Q和K的点积
17     attention = Q @ K.transpose(-2,-1)
18
19     # 缩放
20     attention = attention / (self.head_size ** 0.5)
21     attention = torch.softmax(attention, dim=-1)
22     attention = attention @ V
23
24     return attention

```

Scaled Dot-Product Attention



Multi-Head Attention

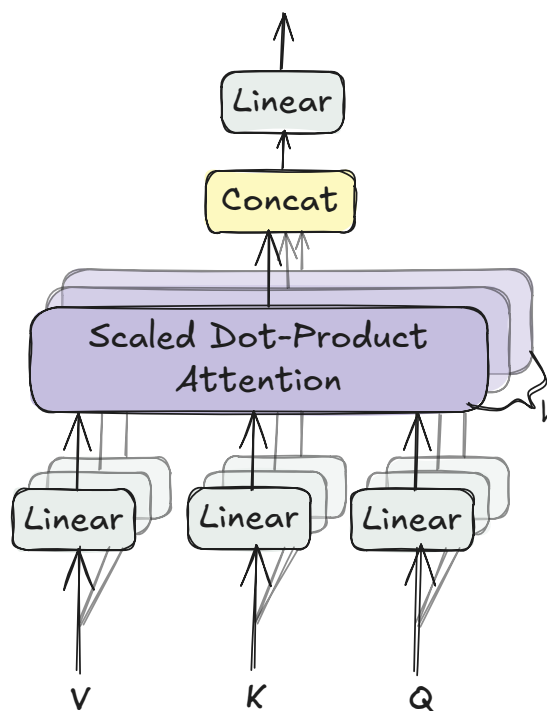


图 1.8 注意力机制

ViT 使用注意力机制，这是一种通信机制，允许模型专注于图像的重要部分。可以使用下面的公式计算注意力分数。

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1.2)$$

计算注意力的第一步是获取 token 的 Q、K 和 V。token 的 Q 是 token 要查找的内容，K 是 token 包含的内容，V 是 token 之间的互信息。Q、K 和 V 可以通过线性层传递 token 来计算。

```

1 def forward(self, x):
2     # 计算Q, K, V
3     Q = self.query(x)

```

Python

```
4 K = self.key(x)
5 V = self.value(x)
```

我们能够通过计算 Q 和 K 的点积来获取序列中 token 之间的关系。

```
1 # Q和K的点积
2 attention = Q @ K.transpose(-2,-1)
```

 Python

我们需要缩放这些值以控制初始化时的方差，以便 token 能够聚合来自多个其他 token 的信息。通过将点积除以注意力头大小的平方根来应用缩放。

```
1 attention = attention / (self.head_size ** 0.5)
```

 Python

然后，我们需要对缩放的点积应用 softmax。

```
1 attention = torch.softmax(attention, dim=-1)
```

 Python

最后，我们需要得到 softmax 和 V 矩阵之间的点积。这本质上是在相应的 token 之间传递信息。

```
1 attention = attention @ V
2 return attention
```

 Python

1.5 多头注意力

多头注意力

 Python

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, d_model, n_heads):
3         super().__init__()
4         self.head_size = d_model // n_heads
5         self.W_o = nn.Linear(d_model, d_model)
6         self.heads = nn.ModuleList([
7             AttentionHead(d_model, self.head_size) for _ in range(n_heads)
8         ])
9
10    def forward(self, x):
11        # 拼接多个注意力头
12        out = torch.cat([head(x) for head in self.heads], dim=-1)
13        out = self.W_o(out)
14        return out
```

多头注意力只是并行运行多个头的自注意力并将它们组合起来。我们可以通过将注意力头添加到模块列表中来做到这一点，

```
1 self.heads = nn.ModuleList([
2     AttentionHead(d_model, self.head_size)
3     for _ in range(n_heads)
4 ])
```

 Python

传递输入然后拼接。

```
1 def forward(self, x):
2     # 拼接多个注意力头
3     out = torch.cat([head(x) for head in self.heads], dim=-1)
```

 Python

然后，我们需要将输出传递给另一个线性层。

```
1 out = self.W_o(out)
```

 Python

```
2 return out
```

1.6 Transformer 编码器

Transformer编码器 Python

```

1 class TransformerEncoder(nn.Module):
2     def __init__(self, d_model, n_heads, r_mlp=4):
3         super().__init__()
4         self.d_model = d_model
5         self.n_heads = n_heads
6
7         # 层归一化
8         self.ln1 = nn.LayerNorm(d_model)
9
10        # 多头注意力
11        self.mha = MultiHeadAttention(d_model, n_heads)
12
13        # 层归一化
14        self.ln2 = nn.LayerNorm(d_model)
15
16        # MLP
17        self.mlp = nn.Sequential(
18            nn.Linear(d_model, d_model*r_mlp),
19            nn.GELU(),
20            nn.Linear(d_model*r_mlp, d_model)
21        )
22
23    def forward(self, x):
24        # 第一次层归一化之后的残差
25        out = x + self.mha(self.ln1(x))
26        # 第二次层归一化之后的残差
27        out = out + self.mlp(self.ln2(out))
28        return out

```

Transformer 编码器由两个子层组成：第一个子层执行多头注意力，第二个子层包含 MLP。多头注意力子层执行 token 之间的通信，而 MLP 子层允许 token 单独“思考”与它们通信的内容。

层归一化是一种优化技术，可跨其特征独立归一化批处理中的每个输入。对于我们的模型，我们将在每个子层的开头通过层归一化模块传递我们的输入。

第一个层归一化模块 Python

```

1 self.ln1 = nn.LayerNorm(d_model)
2
3 # 第二个层归一化模块
4 self.ln2 = nn.LayerNorm(d_model)

```

MLP 将由两个线性层组成，中间有一个 GELU 层。使用 GELU 代替 RELU，因为它没有 RELU 在零点不可导的限制。

编码器的MLP Python

```

1 self.mlp = nn.Sequential(
2     nn.Linear(width, width*r_mlp),
3     nn.GELU(),
4     nn.Linear(width*r_mlp, width)

```

```
6    )
```

在编码器的 `forward` 方法中，输入在执行多头注意力之前通过第一层归一化模块。通过执行多头注意力将原始输入添加到输出中，以创建残差连接。

然后，在输入 MLP 之前，它会通过另一个层归一化模块。通过将 MLP 的输出添加到第一个残差连接的输出，创建另一个残差连接。

残差连接用于通过创建一条路径来帮助防止梯度消失问题，以便梯度不受阻碍地反向传播回原始输入。

```
1 def forward(self, x):
2     # 残差
3     out = x + self.mha(self.ln1(x))
4     # 残差
5     out = out + self.mlp(self.ln2(out))
6     return out
```

 Python

1.7 Vision Transformer

ViT类

 Python

```
1 class VisionTransformer(nn.Module):
2     def __init__(
3         self,
4         d_model,
5         n_classes,
6         img_size,
7         patch_size,
8         n_channels,
9         n_heads,
10        n_layers
11    ):
12        super().__init__()
13
14        assert img_size[0] % patch_size[0] == 0 \
15            and img_size[1] % patch_size[1] == 0, \
16            "img_size 必须能被 patch_size 整除"
17        assert d_model % n_heads == 0, \
18            "d_model 必须能被 n_heads 整除"
19
20        self.d_model = d_model # 模型维度，嵌入的维度（宽度）
21        self.n_classes = n_classes # 类别的数量
22        self.img_size = img_size # 图片大小
23        self.patch_size = patch_size # 补丁大小
24        self.n_channels = n_channels # 通道数
25        self.n_heads = n_heads # 注意力头的数量
26        # 补丁的数量 = (32x32) // (4x4)
27        self.n_patches = (self.img_size[0] * self.img_size[1]) \
28            // (self.patch_size[0] * self.patch_size[1])
29        # 序列的长度 = 1（分类token） + 补丁的数量
30        self.max_seq_length = self.n_patches + 1
31        # 补丁嵌入
32        self.patch_embedding = PatchEmbedding(
```

```

33         self.d_model,
34         self.img_size,
35         self.patch_size,
36         self.n_channels
37     )
38     # 位置编码
39     self.positional_encoding = PositionalEncoding(
40         self.d_model,
41         self.max_seq_length
42     )
43     self.transformer_encoder = nn.Sequential(*[
44         TransformerEncoder(self.d_model, self.n_heads)
45         for _ in range(n_layers)
46     ])
47
48     # 用于分类的MLP
49     self.classifier = nn.Sequential(
50         nn.Linear(self.d_model, self.n_classes),
51         nn.Softmax(dim=-1)
52     )
53
54     def forward(self, images):
55         # 将图片转换成补丁的嵌入 (embedding)
56         x = self.patch_embedding(images)
57         # 添加位置编码
58         x = self.positional_encoding(x)
59         # 编码
60         x = self.transformer_encoder(x)
61         # 分类的线性层
62         x = self.classifier(x[:,0])
63         return x

```

在创建我们的 VisionTransformer 类时，我们首先需要确保输入图像可以均匀地分成大小为 patch_size 的补丁，并且模型的维数可以被注意力头的数量整除。

```

1  assert img_size[0] % patch_size[0] == 0 \
2      and img_size[1] % patch_size[1] == 0, \
3      "img_size 必须能被 patch_size 整除"
4  assert d_model % n_heads == 0, \
5      "d_model 必须能被 n_heads 整除"

```

 Python

我们还需要计算位置编码的最大序列长度，该长度等于补丁的数量加 1。补丁的数量可以通过将输入图像的高和宽的乘积除以补丁大小的高和宽的乘积来计算。

```

1  self.n_patches = (self.img_size[0] * self.img_size[1]) \
2      // (self.patch_size[0] * self.patch_size[1])
3  self.max_seq_length = self.n_patches + 1

```

 Python

VisionTransformer 还需要能够拥有多个编码器模块。这可以通过将编码器层列表放入顺序包装器中来实现。

```

1  self.encoder = nn.Sequential(*[
2      TransformerEncoder(self.d_model, self.n_heads) for _ in range(n_layers)

```

 Python


```
3 ])
```

VisionTransformer 模型的最后一部分是 MLP 分类头。它由一个线性层和一个 softmax 层组成。

```
1 self.classifier = nn.Sequential(  
2     nn.Linear(self.d_model, self.n_classes),  
3     nn.Softmax(dim=-1)  
4 )
```

 Python

在 forward 方法中，输入图像首先经过补丁嵌入层，将图像分割成补丁，并获取这些补丁的线性嵌入序列。然后，它们经过位置编码层，添加分类 token 和位置编码，最后经过编码器模块。之后，分类 token 再经过分类 MLP，以确定图像的分类。

```
1 def forward(self, images):  
2     x = self.patch_embedding(images)  
3     x = self.position_embedding(x)  
4     x = self.encoder(x)  
5     x = self.classifier(x[:,0])  
6     return x
```

 Python

我们已经完成了模型的构建。现在我们需要对其进行训练和测试。

1.8 训练参数

```
1 d_model = 9 # 嵌入的维度9  
2 n_classes = 10 # 类别数量为10  
3 img_size = (32,32) # 图片大小为32x32  
4 patch_size = (16,16) # 补丁的大小是16x16  
5 n_channels = 1 # 灰度图片通道数量为1  
6 n_heads = 3 # 3个注意力头  
7 n_layers = 3 # 3层编码器  
8 batch_size = 128 # 每个批次128张图片  
9 epochs = 5 # 训练5个epoch  
10 alpha = 0.005 # 学习率5e-3
```

 Python

1.9 加载 MNIST 数据集

```
1 transform = T.Compose([  
2     T.Resize(img_size), # 28x28 --> 32x32  
3     T.ToTensor() # 转换成torch.tensor  
4 ])  
5  
6 train_set = MNIST(  
7     root="datasets", train=True, download=True, transform=transform  
8 )  
9 test_set = MNIST(  
10     root="datasets", train=False, download=True, transform=transform  
11 )  
12  
13 train_loader = DataLoader(train_set, shuffle=True, batch_size=batch_size)  
14 test_loader = DataLoader(test_set, shuffle=False, batch_size=batch_size)
```

 Python

1.10 训练循环

下面的代码使用 MNIST 训练集训练我们的 VisionTransformer 类，并展示了整个 epoch 的平均损失。

```
1  device = torch.device("cuda")
2
3  ViT = VisionTransformer(
4      d_model,
5      n_classes,
6      img_size,
7      patch_size,
8      n_channels,
9      n_heads,
10     n_layers
11 ).to(device)
12
13 optimizer = Adam(ViT.parameters(), lr=alpha)
14 criterion = nn.CrossEntropyLoss()
15
16 for epoch in range(epochs):
17     training_loss = 0.0
18     for i, data in enumerate(train_loader, 0):
19         # 取出图像和对应的标签
20         inputs, labels = data
21         inputs, labels = inputs.to(device), labels.to(device)
22
23         optimizer.zero_grad()
24         # 前向传播
25         outputs = ViT(inputs)
26         # 交叉熵损失
27         loss = criterion(outputs, labels)
28         # 求导数
29         loss.backward()
30         # 梯度下降
31         optimizer.step()
32
33         training_loss += loss.item()
34
35     print(f"Epoch {epoch + 1}/{epochs} loss: \
36         {training_loss / len(train_loader) :.3f}")
```

1.11 评估模型

```
1  correct = 0
2  total = 0
3
4  with torch.no_grad():
5      for data in test_loader:
6          images, labels = data
7          images, labels = images.to(device), labels.to(device)
```

```
8     outputs = ViT(images)
9     _, predicted = torch.max(outputs.data, 1)
10    total += labels.size(0)
11    correct += (predicted == labels).sum().item()
12    print(f"\n预测准确率: {100 * correct // total} %")
```

使用此模型，我们仅用 5 个 epoch 就能够在 MNIST 数据集上实现约 92% 的准确率。此示例展示了自注意力机制可以替代深度卷积网络。

2. CLIP

计算机视觉系统历来仅限于一组固定的类别,CLIP是一场革命,它允许通过“预测哪些图像和文本配对在一起”来识别开放世界中的对象。CLIP 能够通过学习批量训练数据的图像和文本特征之间的余弦相似度来预测这一点。这在下图的对比预训练部分显示,其中图像之间的点积特征 $\{I_1, I_2, \dots, I_N\}$ 和文本特征 $\{T_1, T_2, \dots, T_N\}$ 被占用。

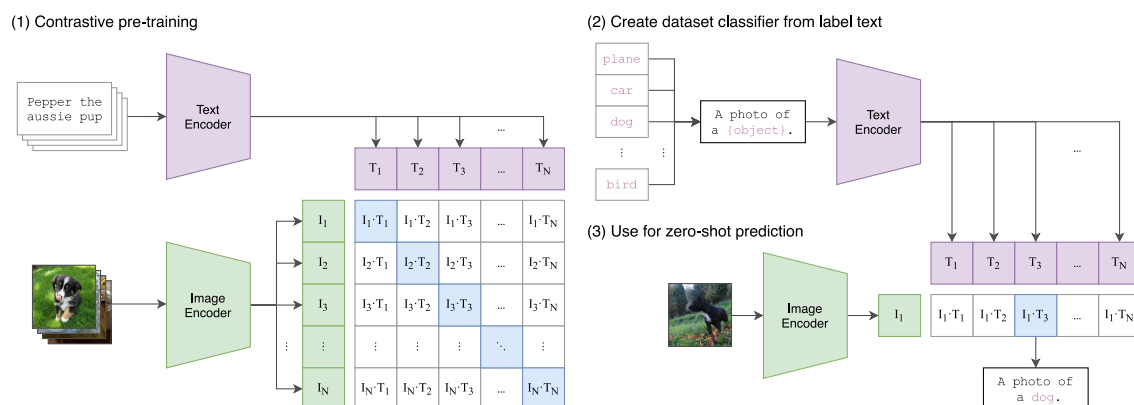


图 2.1 clip 原理, 来自原始论文

在本章中, 我们将从头开始构建 CLIP 并在 MNIST 数据集上对其进行测试。

2.1 导入库和模块

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import torchvision.transforms as T
5 from torch.utils.data import Dataset, DataLoader
6 from datasets import load_dataset
7 import matplotlib.pyplot as plt
8 import numpy as np
```

Python

2.2 图像和文本编码器

我们将首先构建图像和文本编码器。两者分别将图像和文本嵌入到单个 **token** 中，然后可用于对比损失计算。

2.2.1 位置嵌入

```
1 class PositionalEmbedding(nn.Module):
2     def __init__(self, width, max_seq_length):
3         super().__init__()
4         # 创建一个 (token的数量, 嵌入的维度) 形状的全0张量
5         # width 就是 d_model
6         pe = torch.zeros(max_seq_length, width)
7         # 将位置编码信息填充到pe中
8         for pos in range(max_seq_length):
9             for i in range(width):
10                 if i % 2 == 0:
11                     pe[pos][i] = np.sin(pos/(10000 ** (i/width)))
12                 else:
13                     pe[pos][i] = np.cos(pos/(10000 ** ((i-1)/width)))
14         # 位置编码信息进行冻结, 不参与反向传播
15         self.register_buffer('pe', pe.unsqueeze(0))
16
17     def forward(self, x):
18         # 直接将位置编码和token的嵌入进行相加
19         x = x + self.pe
20         return x
```

2.2.2 注意力头

```
1 class AttentionHead(nn.Module):
2     def __init__(self, width, head_size):
3         super().__init__()
4         self.head_size = head_size
5
6         self.query = nn.Linear(width, head_size)
7         self.key = nn.Linear(width, head_size)
8         self.value = nn.Linear(width, head_size)
9
10    def forward(self, x, mask=None):
11        # 计算K, Q, V
12        Q = self.query(x)
13        K = self.key(x)
14        V = self.value(x)
15
16        # Q和K的点积
17        attention = Q @ K.transpose(-2, -1)
18        # 缩放
19        attention = attention / (self.head_size ** 0.5)
20        # 掩码
21        if mask is not None:
```

```

22         attention = attention.masked_fill(mask == 0, float("-inf"))
23         attention = torch.softmax(attention, dim=-1)
24         attention = attention @ V
25         return attention

```

Transformer 编码器和解码器之间的主要区别在于解码器使用注意力掩码，而编码器则不使用。虽然 CLIP 是仅编码器模型 (Encoder-Only)，但由于在分词时，对输入文本添加了 pad (填充符)，所以仍然需要与文本编码器一起使用掩码。请注意，掩码是可选的，因此这个注意力头可用于文本和视觉编码器。

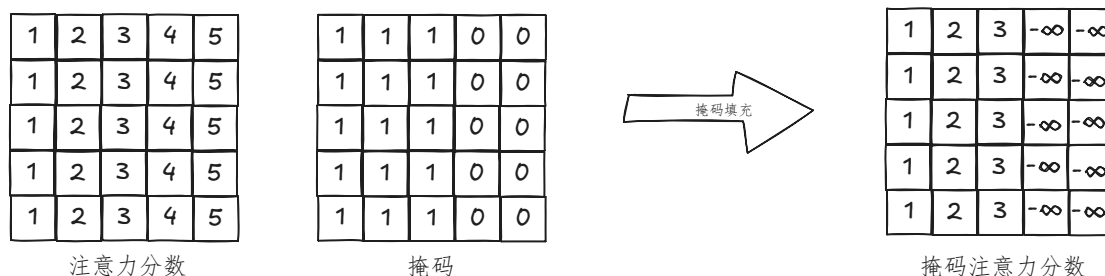


图 2.2 注意力分数的掩码

```

1 # 使用注意力掩码
2 if mask is not None:
3     attention = attention.masked_fill(mask == 0, float("-inf"))

```

Python

2.2.3 多头注意力

```

1 class MultiHeadAttention(nn.Module):
2     def __init__(self, width, n_heads):
3         super().__init__()
4         self.head_size = width // n_heads
5         self.W_o = nn.Linear(width, width)
6         self.heads = nn.ModuleList([
7             AttentionHead(width, self.head_size) for _ in range(n_heads)
8         ])
9
10    def forward(self, x, mask=None):
11        # 拼接多个注意力头
12        out = torch.cat([head(x, mask=mask) for head in self.heads], dim=-1)
13        out = self.W_o(out)
14        return out

```

Python

2.2.4 Transformer 编码器

```

1 class TransformerEncoder(nn.Module):
2     def __init__(self, width, n_heads, r_mlp=4):
3         super().__init__()
4         self.width = width # 嵌入的维度 d_model
5         self.n_heads = n_heads
6

```

Python

```

7         # 层归一化
8         self.ln1 = nn.LayerNorm(width)
9
10        # 多头注意力
11        self.mha = MultiHeadAttention(width, n_heads)
12
13        # 层归一化
14        self.ln2 = nn.LayerNorm(width)
15
16        # MLP
17        self.mlp = nn.Sequential(
18            nn.Linear(self.width, self.width*r_mlp),
19            nn.GELU(),
20            nn.Linear(self.width*r_mlp, self.width)
21        )
22
23        def forward(self, x, mask=None):
24            x = x + self.mha(self.ln1(x), mask=mask)
25            x = x + self.mlp(self.ln2(x))
26            return x

```

2.2.5 文本的分词器

我们的词汇表是 `ascii` 码，所以 `vocab_size = 256`。

```

1 def tokenizer(text, encode=True, mask=None, max_seq_length=32):
2     if encode:
3         out = chr(2) + text + chr(3) # 添加 SOT token 和 EOT token
4         out = out + "".join([
5             chr(0) for _ in range(max_seq_length-len(out))
6         ]) # 添加Padding
7         out = torch.IntTensor(list(out.encode("utf-8"))) # 对文本进行编码
8         mask = torch.ones(len(out.nonzero()))
9         mask = torch.cat((
10             mask,
11             torch.zeros(max_seq_length - len(mask))
12         )).type(torch.IntTensor)
13     else:
14         # 将input_ids解码为text文本
15         out = [chr(x) for x in text[1:len(mask.nonzero())-1]]
16         out = "".join(out)
17         mask = None
18
19     return out, mask

```

Transformer 无法处理原始文本，因此我们需要做的第一件事是在将输入字符串通过文本编码器之前对其进行分词。

在本教程中，我们将进行一个简单版本的分词，其中我们只使用 UTF-8 编码。为什么使用 UTF-8 编码进行分词？因为我们只会在示例中使用简单的 `ascii` 码文本。对于更复杂的示例，可能需要使用 BPE 分词器。这是因为使用 UTF-8 编码时，词表大小 (`vocab_size`) 为 256，这意味着对于更复杂的示例，可能会有更长的输入序列，由于上下文长度有限，这在计算注意力时效率低下。

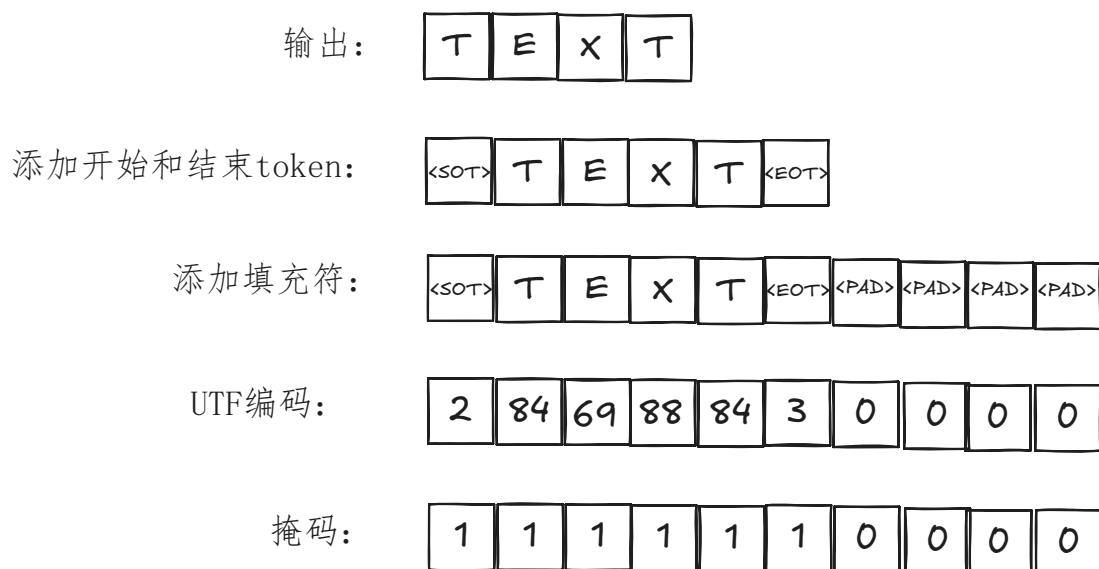


图 2.3 分词过程

分词器的第一步是将文本的开头和文本的结尾 token 添加到输入字符串中。

```
1 text = chr(2) + text + chr(3) Python
```

添加文本的开头和文本的结尾 token 后，我们需要将序列的长度填充到最大序列长度。

```
1 text = text + "".join([chr(0) for _ in range(10-len(text))]) Python
```

我们通过将文本序列编码为 UTF-8 并将输出转换为 IntTensor 来完成分词。

```
1 text = torch.IntTensor(list(text.encode("utf-8"))) Python
```

对文本进行分词后，我们需要为文本创建掩码。虽然 Transformer 中通常使用的掩码用于确保 token 不与未来的 token 通信，但我们在这里应用的掩码只是使 pad token 被忽略。因此，掩码将只是一个大小等于最大序列长度的张量，其中元素在有填充的情况下为 0，否则为 1。

```
1 mask = torch.ones(len(text.nonzero())) Python
2 mask = torch.cat((mask, torch.zeros(10-len(mask)))).type(torch.IntTensor)
```

2.2.6 文本编码器

```
1 class TextEncoder(nn.Module): Python
2     def __init__(
3         self,
4         vocab_size, # 词汇表大小=256
5         width, # 宽度d_model
6         max_seq_length, # 文本最大长度
7         n_heads,
8         n_layers,
9         emb_dim # 嵌入维度
10    ):
11        super().__init__()
12        self.max_seq_length = max_seq_length
13        self.encoder_embedding = nn.Embedding(vocab_size, width)
```



```

14         self.positional_embedding = PositionalEmbedding(
15             width,
16             max_seq_length
17         )
18         self.encoder = nn.ModuleList([
19             TransformerEncoder(width, n_heads)
20             for _ in range(n_layers)
21         ])
22         # 可学习投影 (projection)
23         # d_model(width) --- emb_dim
24         self.projection = nn.Parameter(torch.randn(width, emb_dim))
25
26     def forward(self, text, mask=None):
27         # 文本嵌入
28         x = self.encoder_embedding(text)
29         # 位置嵌入
30         x = self.positional_embedding(x)
31         # Transformer编码器
32         for encoder_layer in self.encoder:
33             x = encoder_layer(x, mask=mask)
34         # 从EOT的嵌入抽取特征
35         x = x[
36             torch.arange(text.shape[0]), # 批次中数据的索引
37             # 取出掩码mask矩阵的第0行, 加和再减1, 就得到了EOT的索引
38             torch.sub(torch.sum(mask[:, 0], dim=1), 1)
39         ]
40         # 将文本特征嵌入到联合嵌入空间中 (多模态嵌入空间)
41         # 文本编码器输出的张量的维度和图像编码器输出的张量的维度必须一致
42         if self.projection is not None:
43             x = x @ self.projection
44
45         x = x / torch.norm(x, dim=-1, keepdim=True)
46         return x

```

对于文本编码器，我们将使用常规 Transformer 模型。创建文本编码器的第一步是创建大小为 (vocab_size, width) 的嵌入表。此嵌入表包含一个向量表示，其大小等于词汇表中每个 token 的 Transformer 模型的 width。

```
1 self.encoder_embedding = nn.Embedding(vocab_size, width)
```

 Python

在输出 Transformer 的结果之前，我们需要将特征嵌入到联合嵌入空间中。我们将通过获取文本特征的点积以及使用 nn.Parameter 创建的可学习的投影来实现这一点。

```
1 # 可学习的投影
```

 Python

```
2 self.projection = nn.Parameter(torch.randn(width, emb_dim))
```

在 forward 方法中，我们要做的第一件事是通过嵌入表传递文本的 token。

```
1 # 文本嵌入
```

 Python

```
2 x = self.encoder_embedding(text)
```

然后，我们需要将位置编码添加到嵌入表的输出中。

```
1 # 位置嵌入
```

 Python

```
2 x = self.positional_embedding(x)
```

添加位置编码后，我们现在可以将其与掩码一起通过编码器层。

```
1 # Transformer编码器
2 for encoder_layer in self.encoder:
3     x = encoder_layer(x, mask=mask)
```

 Python

编码器层的输出是文本的特征。我们将使用从 EOT 的嵌入中抽取的特征。

```
1 # 从EOT的嵌入抽取特征
2 x = x[torch.arange(text.shape[0]),torch.sub(torch.sum(mask[:,0],dim=1),1)]
```

 Python

最后，我们通过计算特征和投影之间的点积，将文本特征嵌入到联合嵌入空间中，并通过除以归一化的点积对其进行归一化。

```
1 # 将文本特征嵌入到联合嵌入空间中（多模态嵌入空间）
2 if self.projection is not None:
3     x = x @ self.projection
4 x = x / torch.norm(x, dim=-1, keepdim=True)
5 return x
```

 Python

2.2.7 图像编码器

```
1 class ImageEncoder(nn.Module):
2     def __init__(
3         self,
4         width, # 补丁嵌入的维度d_model
5         img_size,
6         patch_size,
7         n_channels,
8         n_layers,
9         n_heads,
10        emb_dim
11    ):
12        super().__init__()
13
14        assert img_size[0] % patch_size[0] == 0 \
15            and img_size[1] % patch_size[1] == 0, \
16            "img_size必须能被patch_size整除"
17        assert width % n_heads == 0, \
18            "width必须能被n_heads整除"
19
20        self.n_patches = (img_size[0] * img_size[1]) \
21            // (patch_size[0] * patch_size[1])
22        self.max_seq_length = self.n_patches + 1
23        self.linear_project = nn.Conv2d(
24            n_channels,
25            width,
26            kernel_size=patch_size,
27            stride=patch_size
28        )
29        self.cls_token = nn.Parameter(torch.randn(1, 1, width))
```

 Python

```

30     self.positional_embedding = PositionalEmbedding(
31         width,
32         self.max_seq_length
33     )
34     self.encoder = nn.ModuleList([
35         TransformerEncoder(width, n_heads)
36         for _ in range(n_layers)
37     ])
38
39     # 可学习的投影
40     self.projection = nn.Parameter(torch.randn(width, emb_dim))
41
42     def forward(self, x):
43         # 补丁嵌入
44         x = self.linear_project(x)
45         x = x.flatten(2).transpose(1, 2)
46
47         # 位置嵌入
48         x = torch.cat((self.cls_token.expand(x.size()[0], -1, -1), x), dim=1)
49         x = self.positional_embedding(x)
50
51         # Transformer编码器
52         for encoder_layer in self.encoder:
53             x = encoder_layer(x)
54
55         # 获取类别token
56         x = x[:, 0, :]
57
58         # 多模态嵌入
59         # 保证文本编码器的输出的维度和图像编码器的输出的维度相等
60         if self.projection is not None:
61             x = x @ self.projection
62
63         x = x / torch.norm(x, dim=-1, keepdim=True)
64         return x

```

2.3 CLIP 模型

```

1  class CLIP(nn.Module):
2      def __init__(
3          self,
4          emb_dim, # 经过可学习投影后的嵌入维度
5          vit_width, # 图像编码器的宽度(d_model)
6          img_size,
7          patch_size,
8          n_channels,
9          vit_layers,
10         vit_heads,
11         vocab_size,

```

 Python

```

12         text_width, # 文本编码器的宽度 (d_model)
13         max_seq_length,
14         text_heads,
15         text_layers
16     ):
17         super().__init__()
18         self.image_encoder = ImageEncoder(
19             vit_width,
20             img_size,
21             patch_size,
22             n_channels,
23             vit_layers,
24             vit_heads,
25             emb_dim
26         )
27         self.text_encoder = TextEncoder(
28             vocab_size,
29             text_width,
30             max_seq_length,
31             text_heads,
32             text_layers,
33             emb_dim
34         )
35         # 可学习温度
36         self.temperature = nn.Parameter(torch.ones(1) * np.log(1 / 0.07))
37         self.device = torch.device("cuda")
38
39
40     def forward(self, image, text, mask=None):
41         # I_e是图像嵌入, 形状 [B, D=emb_dim]
42         I_e = self.image_encoder(image)
43         # T_e是文本嵌入, 形状 [B, D=emb_dim]
44         T_e = self.text_encoder(text, mask=mask)
45
46         # 缩放逐点余弦相似度[n, n]
47         # 形状 I_e @ T_e^T : [B, D] @ [D, B] --> [B, B]
48         logits = (I_e @ T_e.transpose(-2, -1)) * torch.exp(self.temperature)
49
50         # 对称损失函数 labels形状为[B], 值为 [0, 1, 2, ..., B-1]
51         labels = torch.arange(logits.shape[0]).to(self.device)
52         # 从文本 --> 图像方向, 以文本嵌入 T_3 为例子,
53         # 交叉熵损失的目标是让 T_3 · I_3 越大越好
54         loss_i = nn.functional.cross_entropy(
55             logits.transpose(-2, -1),
56             labels
57         )
58         # 从图像 --> 文本方向, 以图像嵌入 I_3 为例子,
59         # 交叉熵损失的目标是让 I_3 · T_3 越大越好
60         loss_t = nn.functional.cross_entropy(

```

```

61         logits,
62         labels
63     )
64     # 两个方向的损失求平均值
65     loss = (loss_i + loss_t) / 2
66
67     return loss

```

当给定一批图像和标题时，CLIP 应该告诉我们哪些标题与哪些图像搭配。它通过一起训练文本编码器和图像编码器来最大化应该在一起的对的成对余弦相似度分数，并最小化不应该在一起的对来做到这一点。

为此，我们首先需要从图像和文本编码器中获取特征的嵌入。

```

1 def forward(self, image, text, mask=None):
2     I_e = self.image_encoder(image)
3     T_e = self.text_encoder(text, mask=mask)

```

Python

使用嵌入的特征，我们可以通过使用嵌入的图像特征和嵌入文本特征的转置版本之间的点积来计算缩放的成对余弦相似度。余弦相似度应沿图中正确的图像和文本配对在一起的对角线最大化。

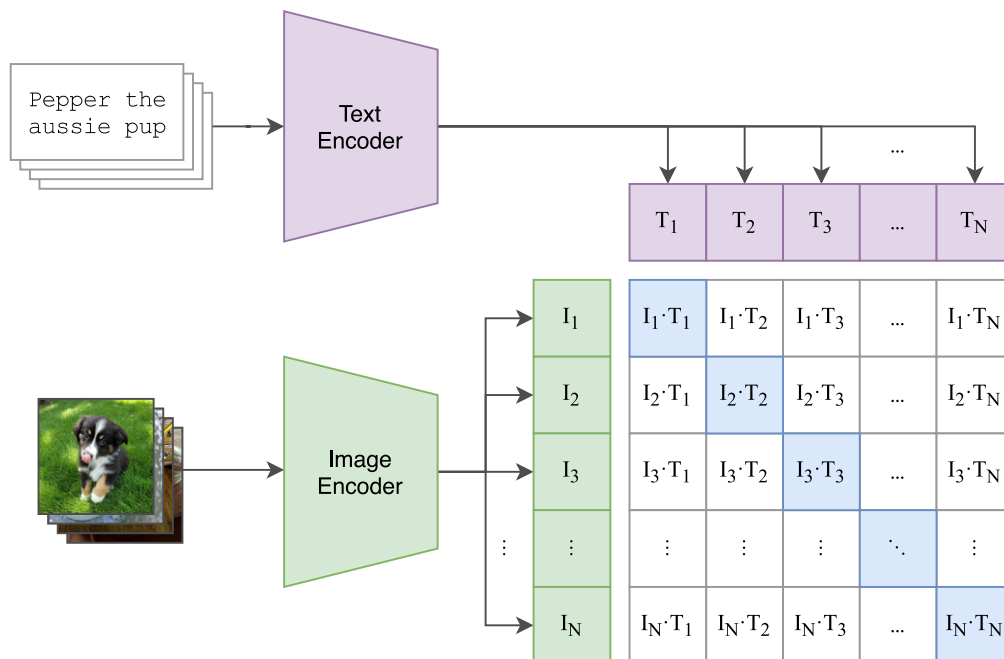


图 2.4 余弦相似度的计算

```

1 logits = (I_e @ T_e.transpose(-2, -1)) * torch.exp(self.temperature)

```

Python

这在 I_e 和 T_e 分别包含 N 个批次时工作，从而产生上图所示的矩阵。为了最大化相关图像之间的余弦相似度，CLIP 使用对称/对比损失。我们可以通过首先创建与批次中的每条数据相对应的标签来计算此损失。

```

1 # 对称损失函数
2 labels = torch.arange(logits.shape[0]).to(self.device)

```

Python

然后，我们计算沿 logit 行的交叉熵损失，以获得图像的损失。

```

1 loss_i = nn.functional.cross_entropy(logits.transpose(-2, -1), labels)

```

Python

文本的损失是通过计算沿列的交叉熵损失来计算的。

```
1 loss_t = nn.functional.cross_entropy(logits, labels)
```

 Python

我们通过计算图像损失和文本损失之间的平均值来获得最终损失。

```
1 loss = (loss_i + loss_t) / 2
2 return loss
```

 Python

2.3.1 数据

```
1 class MNIST(Dataset):
2     def __init__(self, train=True):
3         self.dataset = load_dataset("../datasets/clip-mnist/")
4         self.transform = T.ToTensor()
5         if train:
6             self.split = "train"
7         else:
8             self.split = "test"
9
10        self.captions = {
11            0: "An image of Zero",
12            1: "An image of One",
13            2: "An image of Two",
14            3: "An image of Three",
15            4: "An image of Four",
16            5: "An image of Five",
17            6: "An image of Six",
18            7: "An image of Seven",
19            8: "An image of Eight",
20            9: "An image of Nine"
21        }
22
23    def __len__(self):
24        return self.dataset.num_rows[self.split]
25
26    def __getitem__(self, i):
27        # 取出第i张图片
28        img = self.dataset[self.split][i]["image"]
29        # 转换成张量
30        img = self.transform(img)
31        # 图片对应的文本，以及掩码
32        cap, mask = tokenizer(
33            self.captions[self.dataset[self.split][i]["label"]]
34        )
35        # 为什么要repeat?
36        mask = mask.repeat(len(mask), 1)
37        return {"image": img, "caption": cap, "mask": mask}
```

 Python

在本教程中，我们将使用 MNIST 数据集。我们选择这个数据集是因为它相当小并且保持训练时间合理。

```
1 self.dataset = load_dataset("../datasets/clip-mnist")
```

 Python

对于数据集中的每个条目，我们将需要 3 样东西：图像、文本和文本掩码。

对于图像，我们唯一需要进行的更改是将图像转换为张量。

```
1 img = self.dataset[self.split][i]["image"]
2 img = self.transform(img)
```

 Python

对于标题，我们需要将其传递给我们创建的分词器，以获取 token 表示以及 token 的掩码。

```
1 cap, mask = tokenizer(self.captions[self.dataset[self.split][i]
  ["label"]])
```

 Python

我们从分词器获得的掩码是大小为 `max_seq_length` 的 1 维张量。在文本编码器中，掩码将应用于形状为 `(max_seq_length, max_seq_length)` 的注意力分数。因此，我们需要扩展掩码，以便将其应用于注意力分数的每一行。

原始掩码：

1	1	1	0	0
---	---	---	---	---

新掩码：

1	1	1	0	0
1	1	1	0	0
1	1	1	0	0
1	1	1	0	0
1	1	1	0	0

图 2.5 复制掩码

```
1 mask = mask.repeat(len(mask), 1)
```

 Python

图像、标题和掩码作为字典保存在数据集中。

```
1 return {"image": img, "caption": cap, "mask": mask}
```

 Python

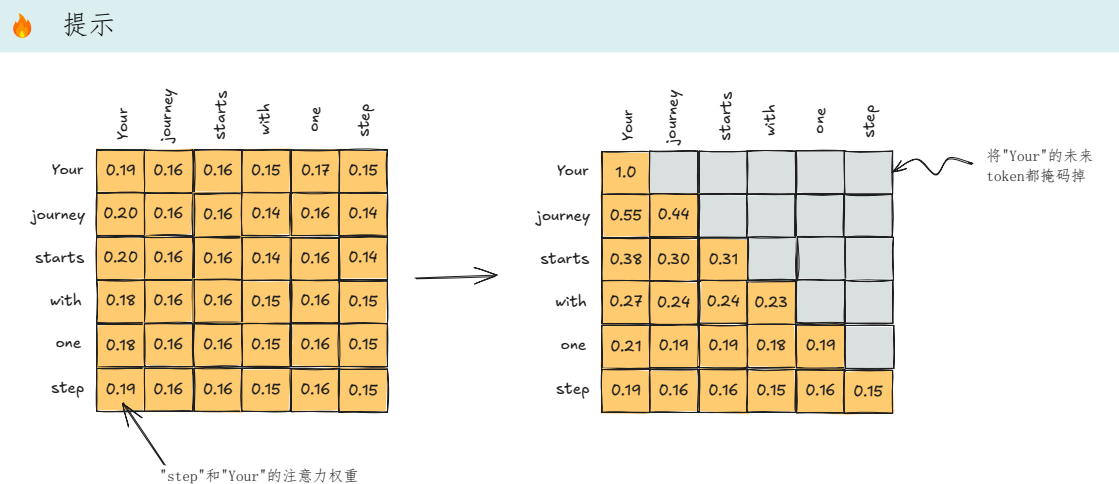


图 2.6 因果注意力分数的掩码

创建 $[\text{context_length}, \text{context_length}]$ 的上三角矩阵。

```
1 self.register_buffer(
2     'mask',
3     torch.triu(torch.ones(
4         context_length,
5         context_length
6     ), diagonal = 1))
```

Python

将注意力分数的上三角置为 $-\text{torch.inf}$ ，下三角保留。得到注意力分数的下三角矩阵。

```
1 attention_score.masked_fill_(
2     self.mask[:seq_len, :seq_len].bool(),
3     -torch.inf
4 )
```

Python

2.3.2 训练参数

```
1 emb_dim = 32 # 文本编码器和图像编码器输出的张量的维度
2 vit_width = 9 # 图像编码器的嵌入的宽度
3 img_size = (28,28)
4 patch_size = (14,14)
5 n_channels = 1
6 vit_layers = 3 # 图像编码器中编码器的层的数量
7 vit_heads = 3 # 图像编码器中注意力头的数量
8 vocab_size = 256 # 词汇表大小
9 text_width = 32 # 文本编码器的嵌入的宽度
10 max_seq_length = 32 # 最大序列长度
11 text_heads = 8 # 文本编码器中注意力头的数量
12 text_layers = 4 # 文本编码器中编码器的层数
13 lr = 1e-3 # 学习率
14 epochs = 10
15 batch_size = 128
```

Python

2.3.3 加载数据集

```
1 train_set = MNIST(train = True)
2 test_set = MNIST(train = False)
3
4 train_loader = DataLoader(train_set, shuffle=True, batch_size=batch_size)
5 test_loader = DataLoader(test_set, shuffle=False, batch_size=batch_size)
```

 Python

2.3.4 训练模型

```
1 device = torch.device("cuda")
2
3 model = CLIP(
4     emb_dim,
5     vit_width,
6     img_size,
7     patch_size,
8     n_channels,
9     vit_layers,
10    vit_heads,
11    vocab_size,
12    text_width,
13    max_seq_length,
14    text_heads,
15    text_layers
16 ).to(device)
17
18 optimizer = optim.Adam(model.parameters(), lr=lr)
19
20 best_loss = np.inf
21 for epoch in range(epochs):
22     for i, data in enumerate(train_loader, 0):
23         img = data["image"].to(device)
24         cap = data["caption"].to(device)
25         mask = data["mask"].to(device)
26         loss = model(img, cap, mask)
27         optimizer.zero_grad()
28         loss.backward()
29         optimizer.step()
30
31     print(f"Epoch [{epoch+1}/{epochs}], Batch Loss: {loss.item():.3f}")
32
33     # 保存模型
34     if loss.item() <= best_loss:
35         best_loss = loss.item()
36         torch.save(model.state_dict(), "./clip.pt")
37     print("模型已经保存.")
```

 Python

2.3.5 评估模型

```

1  # 加载最好的模型
2  model = CLIP(
3      emb_dim,
4      vit_width,
5      img_size,
6      patch_size,
7      n_channels,
8      vit_layers,
9      vit_heads,
10     vocab_size,
11     text_width,
12     max_seq_length,
13     text_heads,
14     text_layers
15 ).to(device)
16 model.load_state_dict(torch.load("./clip.pt", map_location=device))
17
18 # 获取数据集的标签和图片进行对比
19 text = torch.stack(
20     [tokenizer(x)[0] for x in test_set.captions.values()]
21 ).to(device)
22 mask = torch.stack(
23     [tokenizer(x)[1] for x in test_set.captions.values()]
24 )
25 mask = mask.repeat(
26     1,
27     len(mask[0])
28 ).reshape(
29     len(mask),
30     len(mask[0])
31 ), len(mask[0])).to(device)
32
33 correct, total = 0, 0
34 with torch.no_grad():
35     for data in test_loader:
36         # 图像
37         images = data["image"].to(device)
38         # 文本
39         labels = data["caption"].to(device)
40         # 使用clip模型中的图像编码器对图像抽取特征
41         image_features = model.image_encoder(images)
42         # 使用clip模型中的文本编码器对文本抽取特征
43         text_features = model.text_encoder(text, mask=mask)
44         # 归一化
45         image_features /= image_features.norm(dim=-1, keepdim=True)
46         text_features /= text_features.norm(dim=-1, keepdim=True)
47         # I_e @ T_e^T
48         similarity = (

```

```

49         100.0 * image_features @ text_features.T
50     ).softmax(dim=-1)
51     _, indices = torch.max(similarity, 1)
52     # 预测结果
53     pred = torch.stack([
54         tokenizer(test_set.captions[int(i))][0]
55         for i in indices
56     ]).to(device)
57     # 预测正确的样本数量
58     correct += int(sum(torch.sum((pred==labels),dim=1)//len(pred[0])))
59     total += len(labels)
60
61     print(f'\n预测准确率: {100 * correct // total} %')

```

我们通过获取模型训练的标题并将其与实际标题进行比较来测试模型。在训练时，我们使用了相同的标题模板 ("A image of a (n) {class}")，因此这个测试阶段与任何其他图像分类器几乎相同。我们实现了大约 85% 的模型准确率。

2.3.6 零样本分类

```

1  # 加载模型
2  model = CLIP(
3      emb_dim,
4      vit_width,
5      img_size,
6      patch_size,
7      n_channels,
8      vit_layers,
9      vit_heads,
10     vocab_size,
11     text_width,
12     max_seq_length,
13     text_heads,
14     text_layers
15 ).to(device)
16 model.load_state_dict(torch.load("./clip.pt", map_location=device))
17
18 # 标题
19 class_names = [
20     "a photo of 0",
21     "an image of one",
22     "2",
23     "3",
24     "4",
25     "5",
26     "6",
27     "7",
28     "8",
29     "9",
30     "Trump",
31     "Musk"

```

```

32 ]
33
34 text = torch.stack(
35     [tokenizer(x)[0] for x in class_names]
36 ).to(device)
37 mask = torch.stack(
38     [tokenizer(x)[1] for x in class_names]
39 )
40 mask = mask.repeat(
41     1,
42     len(mask[0])
43 ).reshape(len(mask), len(mask[0]), len(mask[0])).to(device)
44
45 idx = 1000
46 # 去测试数据集中的第1000张图片
47 img = test_set[idx]["image"][None,:]
48 plt.imshow(img[0].permute(1, 2, 0), cmap="gray")
49 # 将图片的标题文本展示, 例如 "An Image Of Nine"
50 plt.title(tokenizer(
51     test_set[idx]["caption"],
52     encode=False,
53     mask=test_set[idx]["mask"][0]
54 )[0])
55 plt.show()
56 img = img.to(device)
57 with torch.no_grad():
58     # 抽取图片的特征
59     image_features = model.image_encoder(img)
60     # 抽取文本的特征
61     text_features = model.text_encoder(text, mask=mask)
62
63 image_features /= image_features.norm(dim=-1, keepdim=True)
64 text_features /= text_features.norm(dim=-1, keepdim=True)
65 # 计算第1000张图片和所有文本的相似度
66 similarity = (
67     100.0 * image_features @ text_features.T
68 ).softmax(dim=-1)
69 # 返回所有文本中和图片特征最相似的5个文本
70 values, indices = similarity[0].topk(5)
71
72 # 打印结果
73 print("\n预测结果:\n")
74 for value, index in zip(values, indices):
75     print(f"{class_names[int(index)]:>16s}: {100 * value.item():.2f}%")

```

对于 zero-shot 分类, 我们将图像与类别的名称进行比较。我们输入标签以与图像进行比较, 它将返回前 5 个预测以及预测的可能性。这不是 CLIP 执行 zero-shot 分类的最佳示例。使用 MNIST 数据集使模型易于训练, 但标题不是很丰富。要真正理解 CLIP 的 zero-shot 功能, 包含多个名称的训练集会更适合。真正的 zero-shot 检测将允许检测以前未见过的排列。

 提示

具体步骤如下：

1. 将数据库中的所有图片使用 `clip` 的图像编码器进行抽取特征
2. 将抽取的图片特征存入向量数据库
3. 将输入的搜索文本通过 `clip` 的文本编码器抽取文本特征
4. 使用余弦相似度找出向量数据库中和文本特征最接近的几张图片

3. 扩散模型

扩散模型的兴起可以看作是近年来 AI 生成艺术作品领域取得突破的主要因素。

在图像创作方面，扩散模型已成为内容生成领域的前沿技术。尽管该模型于 2015 年首次推出，但已取得显著进展，并已成为 DALL·E 和 Midjourney 等知名模型的核心机制。



DDPM 算法

Denoising Diffusion Probabilistic Models，去噪扩散概率模型

3.1 通俗讲解

3.1.1 扩散

3.1.1.1 物理学中的类比

想象一下一杯透明的水。如果我们加入少量其他颜色的液体，比如黄色的液体，会发生什么？黄色液体会逐渐均匀地扩散到整个玻璃杯中，最终的混合物会呈现出略带透明的黄色。



图 3.1 水滴的扩散过程

上面的过程被称为 正向扩散：我们通过添加少量其他液体来改变环境状态。然而，进行 反向扩散——将混合物恢复到其原始状态——是否同样容易？事实证明并非如此。即使在最好的情况下，实现这一点也需要高度复杂的机制。

3.1.1.2 将类比应用于机器学习

扩散也可以应用于图像。想象一下一张高质量的狗狗照片。我们可以通过逐渐添加随机噪声来轻松地变换这幅图像。结果，像素值会发生变化，使图像中的狗狗变得不那么明显，甚至无法辨认。这个变换过程称为正向扩散。

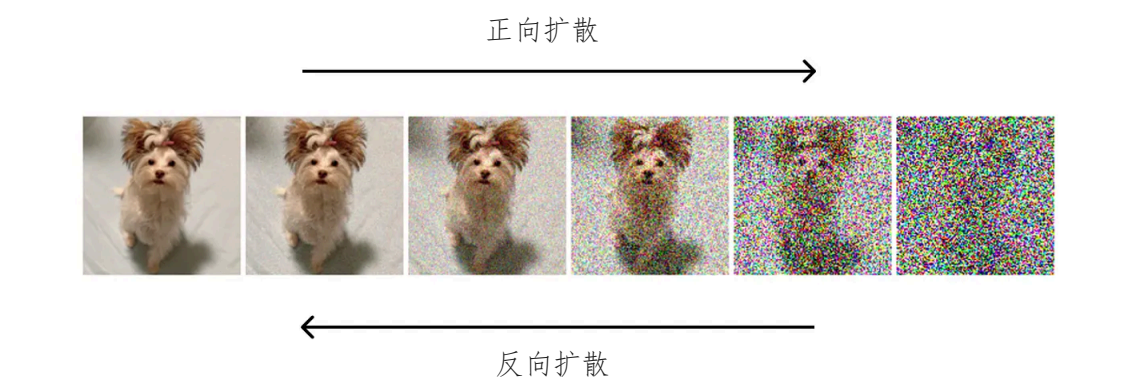


图 3.2 高清图像的扩散过程

我们也可以考虑反向操作：给定一张噪声图像，目标是重建原始图像。这项任务更具挑战性，因为与大量可能的噪声变化相比，可高度识别的图像状态要少得多。用前面提到的物理类比，这个过程称为反向扩散。

在本文中，我将通过示意图来解释它的工作原理。

3.1.2 扩散模型的架构

为了更好地理解扩散模型的结构，让我们分别检查两个扩散过程。

3.1.2.1 正向扩散

如前所述，前向扩散涉及逐步向图像添加噪声。然而，在实践中，这个过程要更加微妙一些。最常见的方法是从均值为 0 的高斯分布中为图片中的每个像素采样一个随机值。然后将这个采样值（可以是正值也可以是负值）添加到像素的原始值中。对所有像素重复此操作会得到原始图像的噪声版本。

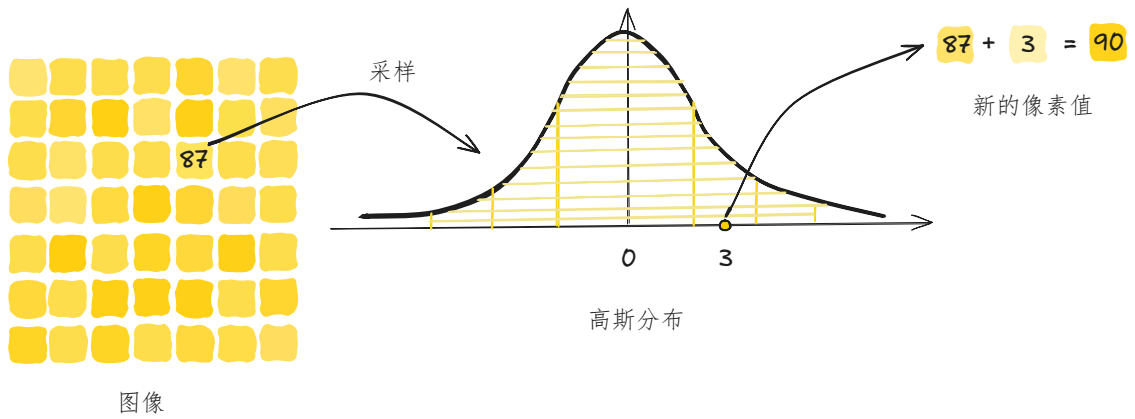


图 3.3 从高斯分布中采样一个随机值

 通知

所选的高斯分布通常方差较小，这意味着采样值通常较小。因此，每一步只会对图像产生微小的变化。
图中有 49 个像素点，那么需要从一个相同的高斯分布中采样 49 次噪声，然后将 49 个噪声分别加到 49 个像素点上。

正向扩散是一个迭代过程，其中噪声被多次应用于图像。随着每次迭代，生成的图像与原始图像的差异越来越大。经过数百次迭代（这在实际扩散模型中很常见）后，图像最终变得无法从纯噪声中识别出来。

3.1.2.2 反向扩散

现在你可能会问：执行所有这些正向扩散变换的目的是什么？答案是，每次迭代生成的图像都用于训练神经网络。

具体来说，假设我们在正向扩散过程中应用了 100 次连续噪声变换。然后，我们可以在每一步获取图像，并训练神经网络重建上一步的图像。预测图像与实际图像之间的差异使用损失函数计算——例如均方误差 (MSE)，它衡量两幅图像之间的平均像素差异。

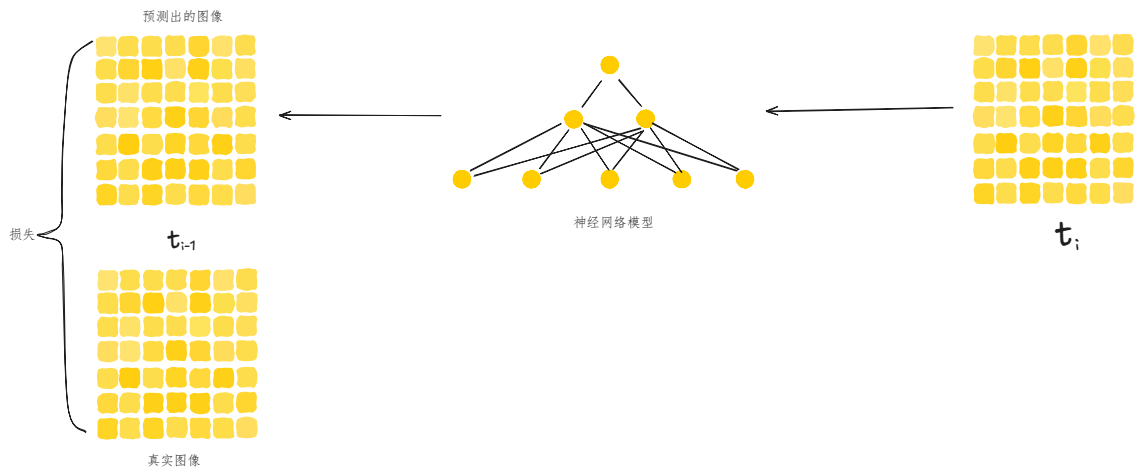


图 3.4 预测图像与实际图像之间的差异使用损失函数计算

 通知

该模型的目标是检测添加的噪声并重建先前的图像。然后将预测图像与实际图像进行比较以计算损失。
这个例子展示了扩散模型重构原始图像的过程。同时，扩散模型可以被训练来预测添加到图像中的噪声。在这种情况下，要重构原始图像，只需要从前一次迭代的图像中减去预测的噪声就足够了。
虽然这两个任务看起来可能相似，但预测添加的噪声比图像重构要简单。

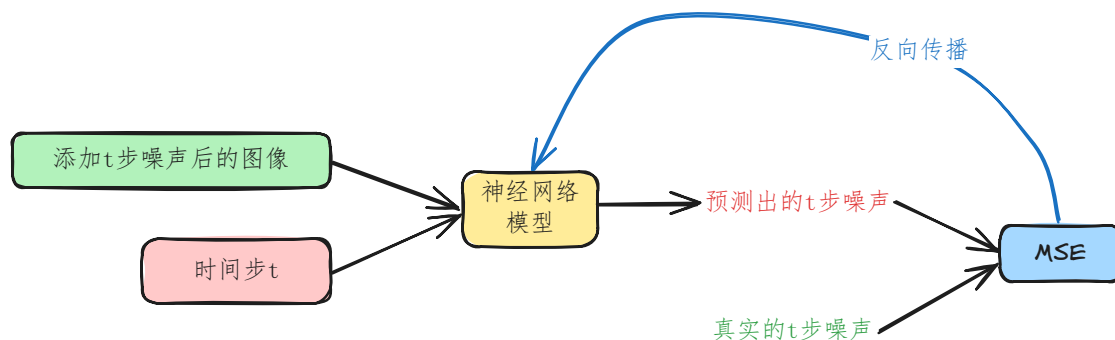


图 3.5 将噪声作为预测目标

3.1.3 模型设计

在对扩散技术有了基本的了解之后，有必要探索一些更高级的概念，以更好地理解扩散模型设计。

3.1.3.1 迭代次数

迭代次数是扩散模型中的关键参数之一：

通知

一方面，使用更多迭代次数意味着相邻步骤中的图像对差异会更小，从而使模型的学习任务更容易。另一方面，更高的迭代次数会增加计算成本。

虽然较少的迭代次数可以加快训练速度，但模型可能无法学习步骤之间的平滑过渡，从而导致性能不佳。

通常，迭代次数选择在 50 到 1000 之间。

3.1.3.2 神经网络架构

最常见的是，U-Net 架构被用作扩散模型的主干。以下是一些原因：

- U-Net 保留了输入和输出图像的尺寸，确保在整个逆扩散过程中图像大小保持一致。
- 其瓶颈架构能够在将整幅图像压缩到潜在空间后将其重建。同时，通过残差连接保留关键图像特征。
- U-Net 最初设计用于生物医学图像分割，其中像素级精度至关重要，它的优势可以很好地转化为需要精确预测单个像素值的扩散任务。

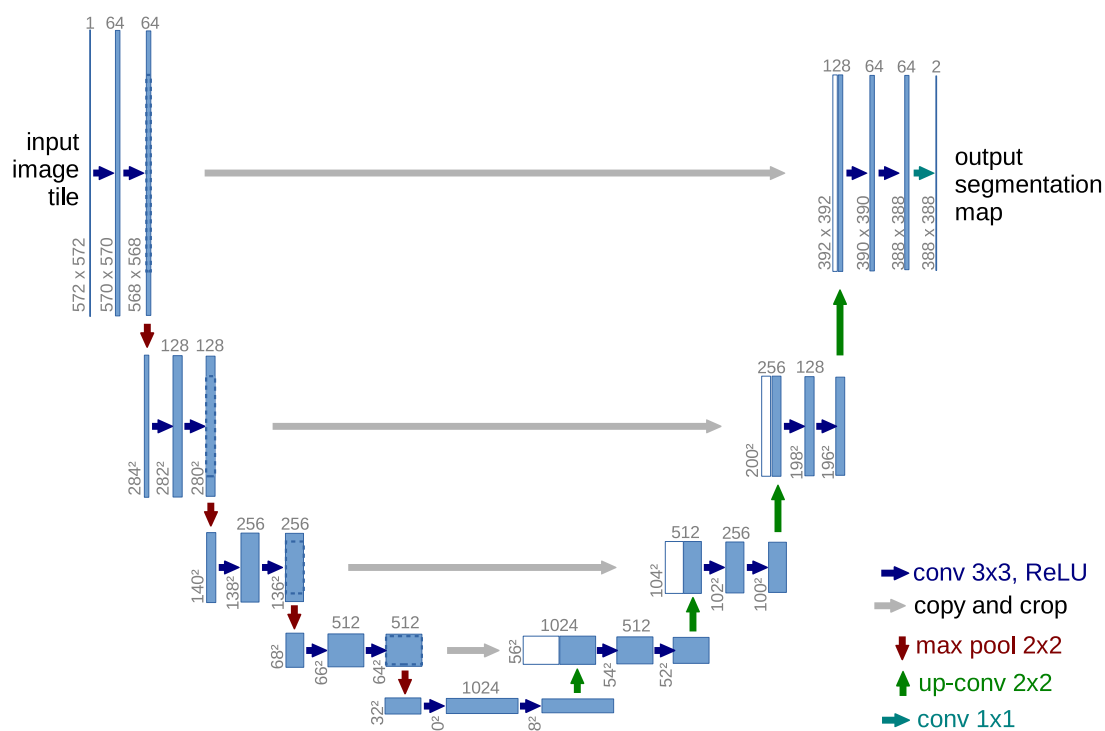


图 3.6 U-net 架构（以最低分辨率为 32×32 像素为例）。每个蓝色方框对应一个多通道特征图。方框顶部标注了通道数量。方框左下角标注了 $x-y$ 尺寸。白色方框表示复制的特征图。箭头表示不同的操作。

3.1.3.3 共享网络

乍一看，似乎有必要为扩散过程的每次迭代训练一个单独的神经网络。虽然这种方法可行，并且可以产生高质量的推理结果，但从计算角度来看，它效率极低。例如，如果扩散过程包含 1000 个时间步，我们就需要训练 1000 个 U-Net 模型——这是一项极其耗时且资源密集的任务。

然而，我们可以观察到，不同迭代中的任务配置本质上是相同的：在每种情况下，我们都需要重建一个尺寸相同、且经过相似幅度噪声改变的图像。这一重要洞见促成了在所有迭代中使用单个共享神经网络的想法。

实际上，这意味着我们使用一个具有共享权重的 U-Net 模型，该模型基于来自不同扩散步骤的图像对进行训练。在推理过程中，含噪图像会多次通过同一个经过训练的 U-Net 模型，逐步进行优化，直到生成高质量的图像。

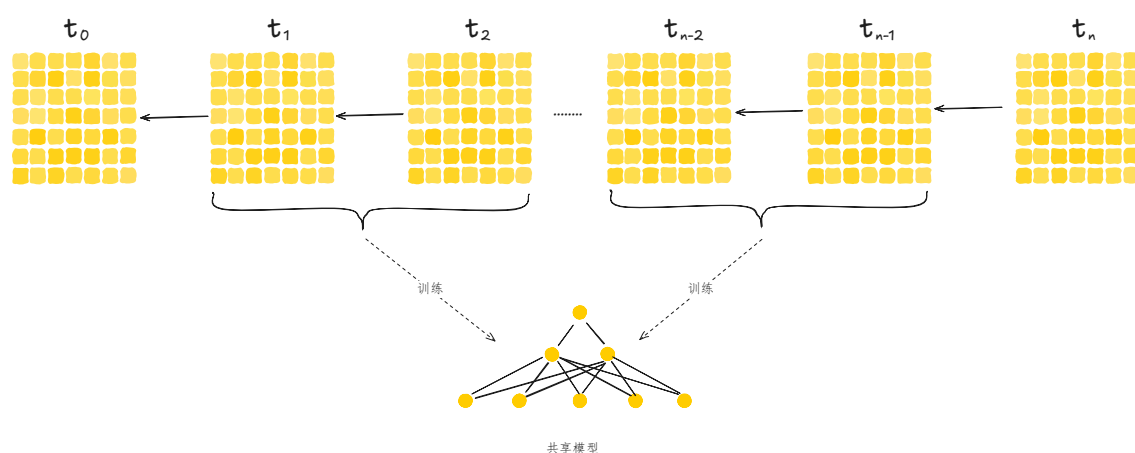


图 3.7 共享模型

虽然由于仅使用单一模型，生成质量可能会略有下降，但训练速度的提升却非常显著。

3.2 扩散模型理论简介

3.2.1 概述

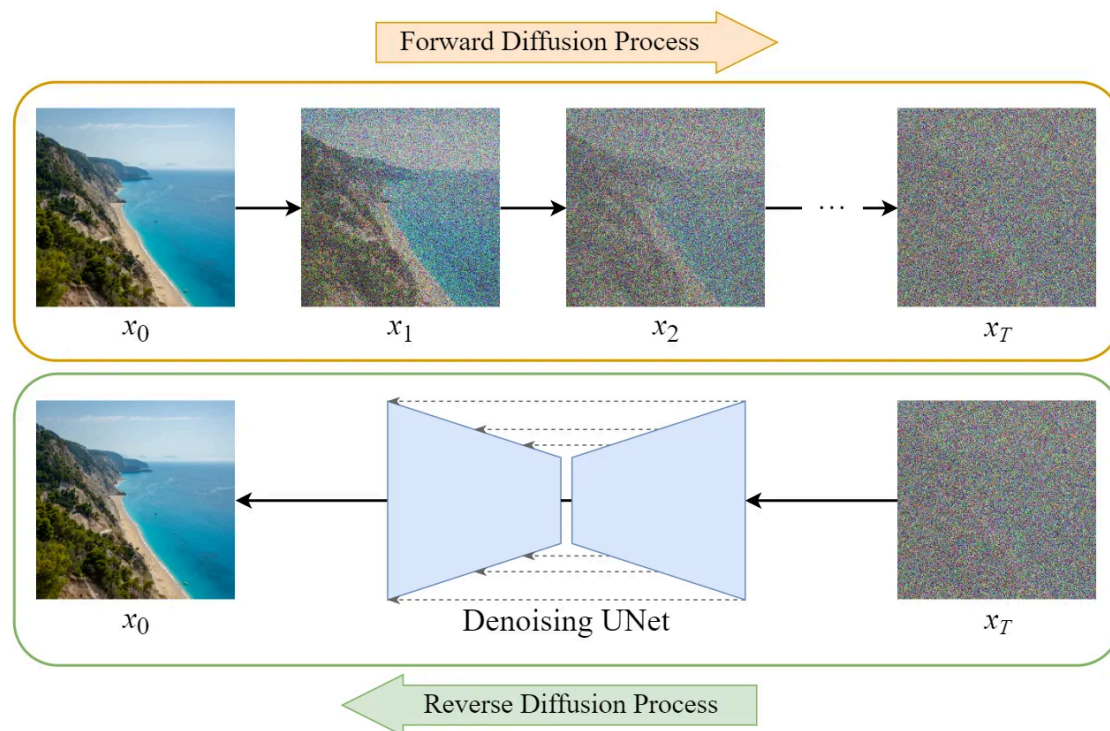


图 3.8 正向扩散和逆向扩散

扩散模型的训练可以分为两部分：

1. 正向扩散过程 $\rightarrow$ 给图像添加噪声。
2. 反向扩散过程 $\rightarrow$ 从图像中去除噪声。

3.2.2 正向扩散过程



4. Sectioning Examples

4.1 Section Title

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.¹

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

4.1.1 Subsection Title

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

4.1.1.1 Subsubsection Title

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

¹Footnote example text...Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent porttitor arcu luctus, imperdiet urna iaculis, mattis eros. Pellentesque iaculis odio vel nisl ullamcorper, nec faucibus ipsum molestie.

feri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Unnumbered Section

Unnumbered Subsection

Unnumbered Subsubsection



5. In-text Element Examples

5.1 Referencing Publications

This statement requires citation [1]; this one is more specific [2, page. 162].

5.2 Link Examples

This is a URL link: [LaTeX Templates](#). This is an email link: example@example.com. This is a monospaced URL link: <https://www.LaTeXTemplates.com>.

5.3 Lists

Lists are useful to present information in a concise and/or ordered way.

5.3.1 Numbered List

1. First numbered item
 - a. First indented numbered item
 - b. Second indented numbered item
 - i. First second-level indented numbered item
 - ii. Second second-level indented numbered item
2. Second numbered item
3. Third numbered item

5.3.2 Bullet Point List

- First bullet point item
 - First indented bullet point item
 - Second indented bullet point item
 - First second-level indented bullet point item
- Second bullet point item
- Third bullet point item

5.3.3 Descriptions and Definitions

Name Definition

Word Definition

Comment Elaboration

5.4 International Support

àáâãäåæèéêëìíîïðóôõöøùúûüýÿñç`cšž
 ĀāĂăĄąĆćĖėĦĥİİıŒœŒœŸŽžŹź
 ßÇÆÆ`CŠŽ

5.5 Ligatures

fi fj fl fl ffi Ty

5.6 Referencing Chapters

This statement references to another chapter Chapter 1.. This statement references to another heading 小节 5.6. This statement references to another heading 小节 6.1.1.

II

Part Two Title

6	Mathematics	59
6.1	Theorems	59
6.2	Definitions	59
6.3	Notations	60
6.4	Remarks	60
6.5	Corollaries	60
6.6	Propositions	60
6.7	Examples	60
6.8	Exercises	60
6.9	Problems	61
6.10	Vocabulary	61
	Presenting Information and Results	
7	with a Long Chapter Title	63
7.1	Table	63
7.2	Figure	63

6. Mathematics

6.1 Theorems

6.1.1 Several equations

This is a theorem consisting of several equations.

Theorem 6.1 – Name of the theorem. In $E = \mathbb{R}^n$ all norms are equivalent. It has the properties:

$$\|x\| - \|y\| \leq \|x - y\| \quad (6.1)$$

$$\left\| \sum_{i=1}^n x_i \right\| \leq \sum_{i=1}^n \|x_i\| \quad \text{where } n \text{ is a finite integer} \quad (6.2)$$

6.1.2 Single Line

This is a theorem consisting of just one line.

Theorem 6.2 A set $\mathcal{D}(G)$ is dense in $L^2(G)$, $|\cdot|_0$.

6.2 Definitions

A definition can be mathematical or it could define a concept.

定义 6.1 – Definition name. Given a vector space E , a norm on E is an application, denoted $\|\cdot\|$, E in $\mathbb{R}^+ = [0, +\infty[$ such that:

$$\|x\| = 0 \Rightarrow x = 0 \quad (6.3)$$

$$\|\lambda x\| = |\lambda| \cdot \|x\| \quad (6.4)$$

$$\|x + y\| \leq \|x\| + \|y\| \quad (6.5)$$

6.3 Notations

Notation 6.1 Given an open subset G of $\mathbf{R}^n$, the set of functions φ are:

1. Bounded support G ;
 2. Infinitely differentiable;
- a vector space is denoted by $\mathcal{D}(G)$.

6.4 Remarks

This is an example of a remark.



The concepts presented here are now in conventional employment in mathematics. Vector spaces are taken over the field $\mathbb{K} = \mathbb{R}$, however, established properties are easily extended to $\mathbb{K} = \mathbb{C}$.

6.5 Corollaries

Corollary 6.1 – Corollary name. The concepts presented here are now in conventional employment in mathematics. Vector spaces are taken over the field $\mathbb{K} = \mathbb{R}$, however, established properties are easily extended to $\mathbb{K} = \mathbb{C}$.

6.6 Propositions

6.6.1 Several equations

Proposition 6.1 – Proposition name. It has the properties:

$$\|x\| - \|y\| \leq \|x - y\| \quad (6.6)$$

$$\left\| \sum_{i=1}^n x_i \right\| \leq \sum_{i=1}^n \|x_i\| \quad \text{where } n \text{ is a finite integer} \quad (6.7)$$

6.6.2 Single Line

Proposition 6.2 Let $f, g \in L^2(G)$; if $\forall \varphi \in \mathcal{D}(G)$, $(f, \varphi)_0 = (g, \varphi)_0$ then $f = g$.

6.7 Examples

6.7.1 Equation Example



例子

Let $G = \{x \in \mathbb{R}^2 : |x| < 3\}$ and denoted by: $x^0 = (1, 1)$; consider the function:

$$f(x) = \begin{cases} e^{|x|} & \text{si } |x - x^0| \leq 1/2 \\ 0 & \text{si } |x - x^0| > 1/2 \end{cases} \quad (6.8)$$

The function f has bounded support, we can take $A = \{x \in \mathbb{R}^2 : |x - x^0| \leq 1/2 + \varepsilon\}$ for all $\varepsilon \in]0; 5/2 - \sqrt{2}[$.

6.7.2 Text Example

6.8 Exercises

Exercise 6.1 This is a good place to ask a question to test learning progress or further cement ideas into students' minds.

6.9 Problems

Problem 6.1 What is the average airspeed velocity of an unladen swallow?

6.10 Vocabulary

Define a word to improve a students' vocabulary.

■ **Vocabulary 6.1 — Word.** Definition of word.

7. Presenting Information and Results with a Long Chapter Title

7.1 Table

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent porttitor arcu luctus, imperdiet urna iaculis, mattis eros. Pellentesque iaculis odio vel nisl ullamcorper, nec faucibus ipsum molestie. Sed dictum nisl non aliquet porttitor. Etiam vulputate arcu dignissim, finibus sem et, viverra nisl. Aenean luctus congue massa, ut laoreet metus ornare in. Nunc fermentum nisi imperdiet lectus tincidunt vestibulum at ac elit. Nulla mattis nisl eu malesuada suscipit.

Treatments	Response 1	Response 2
Treatment 1	0.0003262	0.562
Treatment 2	0.0015681	0.910
Treatment 3	0.0009271	0.296

表 7.1 Table caption.

Referencing 表 7.1 in-text using its label.

7.2 Figure

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Praesent porttitor arcu luctus, imperdiet urna iaculis, mattis eros. Pellentesque iaculis odio vel nisl ullamcorper, nec faucibus ipsum molestie. Sed dictum nisl non aliquet porttitor. Etiam vulputate arcu dignissim, finibus sem et, viverra nisl. Aenean luctus congue massa, ut laoreet metus ornare in. Nunc fermentum nisi imperdiet lectus tincidunt vestibulum at ac elit. Nulla mattis nisl eu malesuada suscipit.



图 7.1 Figure caption.

Treatments	Response 1	Response 2
Treatment 1	0.0003262	0.562
Treatment 2	0.0015681	0.910
Treatment 3	0.0009271	0.296

表 7.2 Floating table.

Referencing 图 7.1 in-text using its label and referencing 图 7.2 in-text using its label.



图 7.2 Floating figure.

参考文献

- [1] A. Jones 和 J. Smith, 《Article Title》, *Journal title*, 卷 13, 期 52, 页 123~456, 2022, doi: [10.1038/s41586-021-03616-x](#).
- [2] J. Smith 和 A. Jones, *Book Title*, 7th 版. Publisher, 2021.

Index

C

Citation	55
Corollaries	60

D

Definitions	59
-----------------------	----

E

Examples	60
Equation	60
Text	60
Exercises	60

F

Figure	63
------------------	----

L

Links	55
Lists	55
Bullet Points	55
Descriptions and Definitions	55
Numbered List	55

N

Notations	60
---------------------	----

P

Problems	61
Propositions	60
Several equations	60
Single Line	60

R

Referencing	56
Remarks	60

S

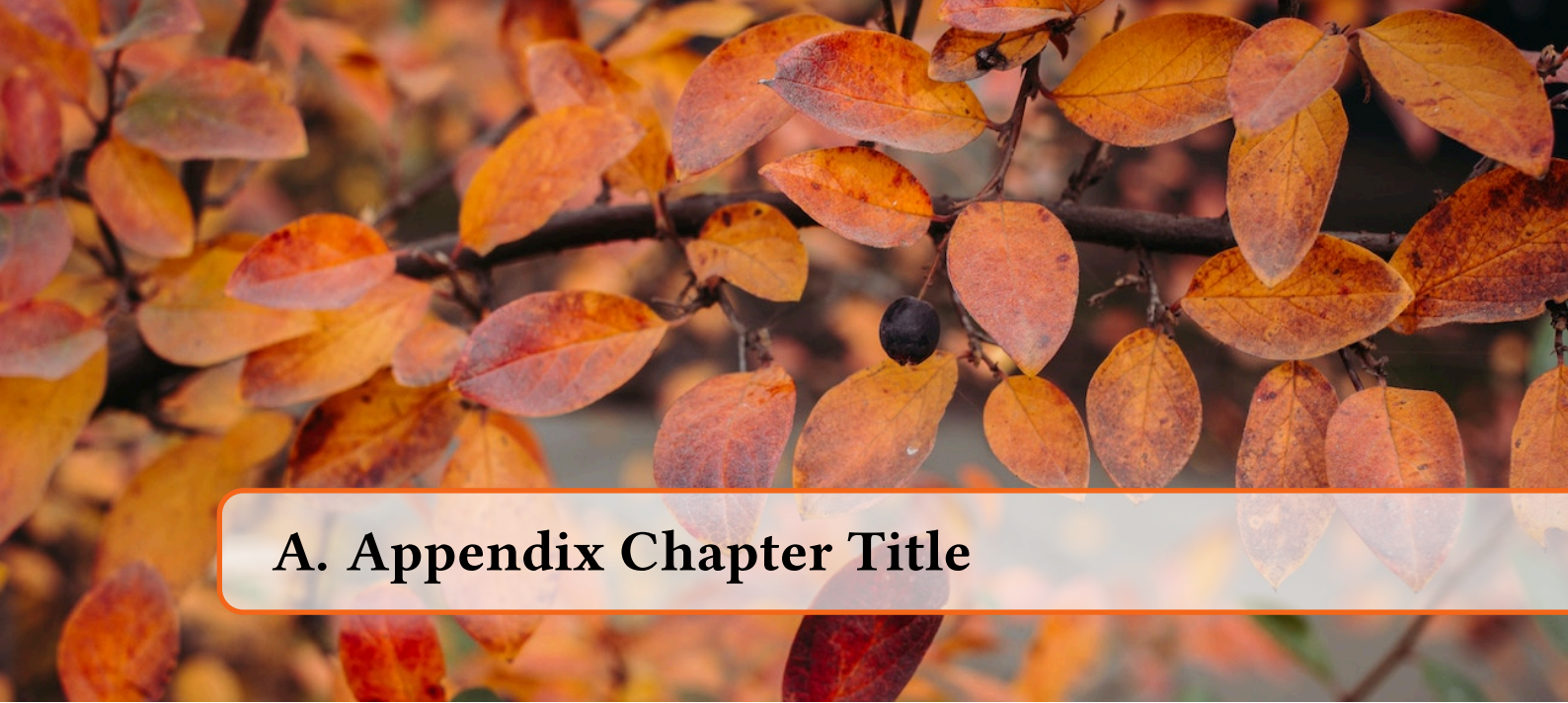
Sectioning	51
Paragraphs	52
Sections	51
Subsections	51
Subsubsections	51

T

Table	63
Theorems	59
Several equations	59
Single Line	59

V

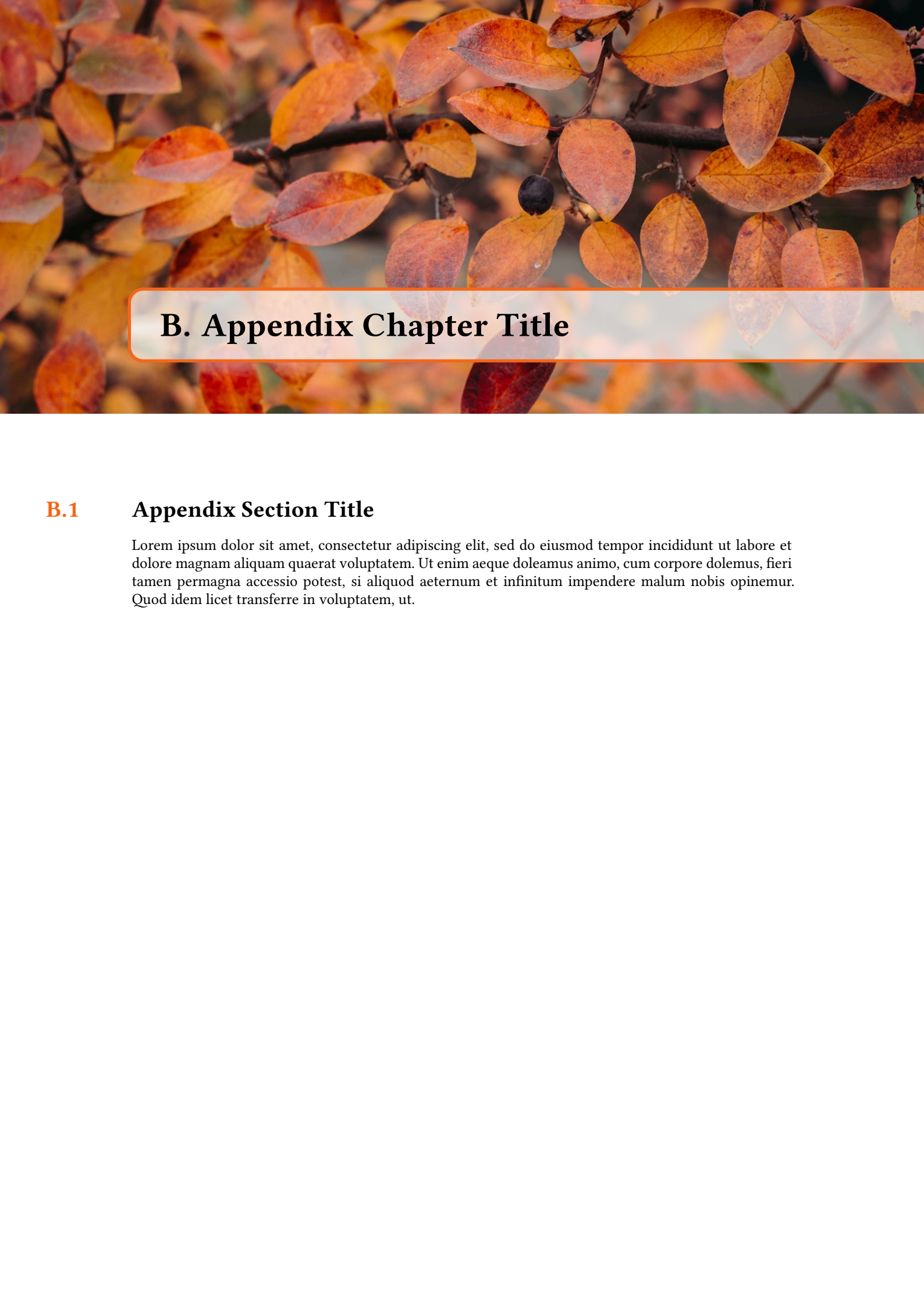
Vocabulary	61
----------------------	----



A. Appendix Chapter Title

A.1 Appendix Section Title

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.



B. Appendix Chapter Title

B.1 Appendix Section Title

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaleam animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.